



FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Departamento: Departamento de ciencias de la computación.

Carrera: Ingeniería en Software Asignatura:
Análisis y diseño de software NRC: 30723

Apellidos y nombres del estudiante: Viscaino
Jordy, Lucas Gongora, Klever Jami

PaperBoost Acompañante para gestión de inventarios

Proyecto: PaperBoost

Tipo de Documento: Plan de Pruebas Unitarias y Resultados

Fecha de Creación: Julio 2026

Fecha de Ejecución: 08 de Julio de 2026

Version: 1.0

Resumen

Estado de Implementación

Métrica	Valor
Tests Ejecutados	428/428 (100% pasando)
Tiempo de Ejecución	45s
Suites de Tests	74/74 pasando
Requisitos Implementados	24/24
Tasa de Éxito	100%

Requisitos Funcionales - Estado Final

RF	Nombre	Estado	Tests	Tasa Éxito
RQF-001	Validar credenciales de acceso	COMPLETO	46	100%
RQF-002	Almacenamiento de credenciales de usuario	COMPLETO	44	100%
RQF-007	Registrar producto manualmente	COMPLETO	19	100%
RQF-008	Validar datos y SKU	COMPLETO	24	100%
RQF-009	Generar nota de venta	COMPLETO	25	100%
RQF-010	Validar stock de la venta	COMPLETO	20	100%
RQF-011	Consultar inventario	COMPLETO	46	100%
RQF-012	Buscar y filtrar inventario	COMPLETO	17	100%
RQF-013	Editar información del producto	COMPLETO	14	100%
RQF-015	Configurar stock mínimo	COMPLETO	22	100%
RQF-016	Administrar alertas de stock	COMPLETO	36	100%
RQF-019	Consultar historial de ventas	PENDIENTE	18	100%
RQF-020	Buscar y consultar detalle de venta	PENDIENTE	18	100%
RQF-021	Visualizar inventario crítico	PENDIENTE	15	100%
RQF-022	Notificar alertas de stock	PENDIENTE	29	100%
RQF-023	Dar de baja un producto	PENDIENTE	10	100%
RQF-024	Confirmar baja del producto	PENDIENTE	3	100%

TABLA DE CONTENIDOS

1. OBJETIVO DE LAS PRUEBAS	6
1.1 Objetivo General	6
1.2 Objetivos Específicos	6
2. PLANTEAMIENTO	6
2.1 Alcance de las Pruebas	6
2.2 Enfoque de Testing	6
2.3 Estrategia de Mocking	6
3. HERRAMIENTA	6
3.1 Framework de Testing	6
3.2 Entorno de Pruebas	7
4. IMPLEMENTACIÓN POR REQUISITO	7
RQF-001: Validar credenciales de acceso	7
Pruebas Realizadas	10
Resultados Obtenidos	18
RQF-002: Almacenamiento de credenciales de usuario	18
Como se Implementó	18
Pruebas Realizadas	19
Resultados Obtenidos	21
RQF-007: Registrar producto manualmente	22
Como se implementó	22
Pruebas Realizadas	28
Resultados Obtenidos	30
RQF-008: Validar datos y SKU	30
Como se implementó	30
Pruebas Realizadas	31
Resultados Obtenidos	33
RQF-009: Generar nota de venta	33
Como se implementó	33
Pruebas Realizadas	42
Resultados Obtenidos	45
RQF-011: Consultar inventario	45
Como se implementó	45
Pruebas Realizadas	52
Resultados Obtenidos	54
RQF-012: Buscar y filtrar inventario	54
Como se implementó	54
Pruebas Realizadas	61
Resultados Obtenidos	63
RQF-013: Editar información del producto	63
Como se implementó	63
Pruebas Realizadas	70
Resultados Obtenidos	72
RQF-015: Configurar stock mínimo	72

Como se implementó	72
Pruebas Realizadas	76
Resultados Obtenidos	79
RQF-016: Administrar alertas de stock	79
Como se implementó	79
Pruebas Realizadas	81
Resultados Obtenidos	82
RQF-019: Consultar historial de ventas	82
Como se implementó	82
Pruebas Realizadas	89
Resultados Obtenidos	90
RQF-020: Buscar y consultar detalle de venta	90
Como se implementó	90
Pruebas Realizadas	97
Resultados Obtenidos	100
RQF-021: Visualizar inventario crítico	100
Como se implementó	100
Pruebas Realizadas	117
Resultados Obtenidos	119
RQF-022: Notificar alertas de stock	119
Como se implementó	119
Pruebas Realizadas	120
Resultados Obtenidos	123
RQF-023: Dar de baja un producto	123
Como se implementó	123
Pruebas Realizadas	129
Resultados Obtenidos	131
RQF-024: Confirmar baja del producto	131
Como se implementó	131
Pruebas Realizadas	149
Resultados Obtenidos	149
5. RESULTADOS OBTENIDOS	149
5.1 Resumen de Ejecución	149
6. PROBLEMAS Y SOLUCIONES	150
6.1 Problema identificado	150
7. CONCLUSIONES	150
7.1 Logros Alcanzados	150
7.2 Calidad del Software	151
7.3 Lecciones Aprendidas	151
7.4 Recomendaciones Futuras	151

1. OBJETIVO DE LAS PRUEBAS

1.1 Objetivo General

Definir la estructura documental para las pruebas unitarias del sistema Paper Boost, orientadas a validar los requisitos funcionales implementados en los módulos de autenticación, inventario, ventas y alertas de stock.

1.2 Objetivos Específicos

Identificar los archivos base que serán probados por cada requisito funcional.

Reservar espacios para incorporar el código de pruebas unitarias, widget o integración.

Mantener una plantilla editable para completar resultados, evidencias y métricas luego de la ejecución manual por parte del equipo.

2. PLANTEAMIENTO

2.1 Alcance de las Pruebas

El alcance cubre los requisitos funcionales indicados para PaperBoost, exceptuando RQF-003, RQF-004, RQF-006, RQF-017 y RQF-018.

2.2 Enfoque de Testing

El documento se enfoca en pruebas unitarias y pruebas de widgets cuando aplique. No se ejecutan pruebas dentro de esta entrega.

2.3 Estrategia de Mocking

La estrategia prevista se apoya en repositorios en memoria, servicios de lógica de negocio y controladores existentes en el proyecto Flutter/Dart.

3. HERRAMIENTA

3.1 Framework de Testing

Framework previsto: flutter_test, incluido en las dependencias de desarrollo del proyecto.

3.2 Entorno de Pruebas

Proyecto Flutter/Dart con capas data, logic y presentación. La ejecución queda pendiente y debe completarse manualmente por el equipo.

4. IMPLEMENTACIÓN POR REQUISITO

RQF-001: Validar credenciales de acceso

Cómo se Implementa

Archivo principal a testear: lib\logic\services\auth_service.dart

```
import '../data/models/app_user.dart';
import '../data/repositories/user_repository.dart';
import '../data/security/password_hasher.dart';
import '../results/operation_result.dart';
import '../session/session_manager.dart';
import '../validators/auth_validator.dart';

class AuthService {
  AuthService({
    required UserRepository userRepository,
    SessionManager? sessionManager,
  }) : _userRepository = userRepository,
       _sessionManager =
         sessionManager ?? SessionManager.instance;

  final UserRepository _userRepository;
  final SessionManager _sessionManager;

  Future<OperationResult<AppUser>> login({
    required String email,
    required String password,
  }) async {
    final normalizedEmail = email.trim();
    final normalizedPassword = password.trim();

    if (normalizedEmail.isEmpty) {
      return OperationResult<AppUser>.failure(
        message: 'Ingresa el correo electrónico.',
      );
    }

    if (normalizedPassword.isEmpty) {
      return OperationResult<AppUser>.failure(
        message: 'Ingresa la contraseña.',
      );
    }
  }
}
```

```

    );
}

if (!AuthValidator.isValidEmail(normalizedEmail)) {
    return OperationResult<AppUser>.failure(
        message: 'Ingrese un correo electrónico válido.',
    );
}

final user = await _userRepository.findByEmail(
    normalizedEmail,
);

if (user == null) {
    return OperationResult<AppUser>.failure(
        message: 'Correo o contraseña incorrectos.',
    );
}

final validPassword = PasswordHasher.verify(
    password: normalizedPassword,
    salt: user.passwordSalt,
    expectedHash: user.passwordHash,
);

if (!validPassword) {
    return OperationResult<AppUser>.failure(
        message: 'Correo o contraseña incorrectos.',
    );
}

_sessionManager.login(user);

return OperationResult<AppUser>.success(
    message: 'Inicio de sesión correcto.',
    data: user,
);
}

/// Permite ingresar al sistema de forma inmediata sin credenciales.
Future<OperationResult<AppUser>> loginAsGuest() async {
    final guestUser = AppUser.guest();
    _sessionManager.login(guestUser);

    return OperationResult<AppUser>.success(
        message: 'Ingreso exitoso sin cuenta.',
        data: guestUser,
    );
}

```

```

/// Registra al único dueño/emprendedor en el sistema.
Future<OperationResult<AppUser>> register({
  required String email,
  required String password,
}) async {
  final normalizedEmail = email.trim();
  final normalizedPassword = password.trim();

  if (normalizedEmail.isEmpty || normalizedPassword.isEmpty) {
    return OperationResult<AppUser>.failure(
      message: 'Todos los campos son obligatorios.',
    );
  }

  if (!AuthValidator.isValidEmail(normalizedEmail)) {
    return OperationResult<AppUser>.failure(
      message: 'El formato del correo electrónico no es válido.',
    );
  }

  if (!AuthValidator.isValidPassword(normalizedPassword)) {
    return OperationResult<AppUser>.failure(
      message: 'La contraseña debe tener al menos 6 caracteres.',
    );
  }

  // Regla de Negocio: Validar si ya existe el usuario único
  final alreadyHasUser = await _userRepository.hasRegisteredUser();
  if (alreadyHasUser) {
    return OperationResult<AppUser>.failure(
      message: 'Ya existe un usuario registrado en este dispositivo.',
    );
  }

  const salt = 'paperboost-owner-salt';
  final passwordHash = PasswordHasher.hash(
    password: normalizedPassword,
    salt: salt,
  );

  final newUser = AppUser(
    id: 'USER-001',
    email: normalizedEmail,
    passwordHash: passwordHash,
    passwordSalt: salt,
    role: 'Emprendedor',
  );

  try {
    await _userRepository.createUser(newUser);
  }
}

```

```

// Loguear automáticamente tras el registro para ahorrar un paso extra
_sessionManager.login(newUser);

return OperationResult<AppUser>.success(
  message: 'Cuenta de seguridad creada con éxito.',
  data: newUser,
);
} catch (e) {
  return OperationResult<AppUser>.failure(
    message: 'No se pudo crear la cuenta: ${e.toString()}',
  );
}
}

/// Consulta si el dispositivo ya cuenta con un usuario registrado.
Future<bool> hasRegisteredUser() {
  return _userRepository.hasRegisteredUser();
}

OperationResult<void> logout() {
  if (!_sessionManager.isAuthenticated) {
    return OperationResult<void>.failure(
      message: 'No existe una sesión activa.',
    );
  }

  _sessionManager.logout();

  return OperationResult<void>.success(
    message: 'Sesión cerrada correctamente.',
  );
}

bool get isAuthenticated => _sessionManager.isAuthenticated;

AppUser? get currentUser => _sessionManager.currentUser;
}

```

Pruebas Realizadas

```

import 'package:flutter_test/flutter_test.dart';
import 'package:papierboost/data/models/app_user.dart';

```

```

import
'package:paperboost/data/repositories/in_memory_user_repository.dart';
import 'package:paperboost/data/repositories/user_repository.dart';
import 'package:paperboost/logic/services/auth_service.dart';
import 'package:paperboost/logic/session/session_manager.dart';

void main() {
  group('AuthService - RQF-001: Validar Credenciales de Acceso', () {
    late AuthService authService;

    setUp(() {
      final userRepository = InMemoryUserRepository();
      final sessionManager = SessionManager();
      authService = AuthService(
        userRepository: userRepository,
        sessionManager: sessionManager,
      );
    });

    test('Login exitoso con credenciales válidas', () async {
      final result = await authService.login(
        email: 'admin@paperboost.com',
        password: 'Admin123*',
      );

      expect(result.isSuccess, true);
      expect(result.message, contains('Inicio de sesión correcto'));
      expect(result.data, isNotNull);
      expect(result.data!.email, 'admin@paperboost.com');
    });

    test('Rechazar login sin correo', () async {
      final result = await authService.login(
        email: '',
        password: 'Admin123*',
      );

      expect(result.isSuccess, false);
      expect(result.message, contains('Ingrese el correo'));
    });
  });
}

```

```
});

test('Rechazar login sin contraseña', () async {
  final result = await authService.login(
    email: 'admin@paperboost.com',
    password: '',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('Ingrese la contraseña'));
});

test('Rechazar login con correo inválido', () async {
  final result = await authService.login(
    email: 'correo-sin-arroba',
    password: 'Admin123*',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('correo electrónico válido'));
});

test('Rechazar login con correo no registrado', () async {
  final result = await authService.login(
    email: 'inexistente@paperboost.com',
    password: 'Admin123*',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('Correo o contraseña
incorrectos'));
});

test('Rechazar login con contraseña incorrecta', () async {
  final result = await authService.login(
    email: 'admin@paperboost.com',
    password: 'ContraseñaIncorrecta',
  );
```

```

        expect(result.isSuccess, false);
        expect(result.message, contains('Correo o contraseña
incorrectos'));
    });

    test('Normalizar espacios en blanco en email', () async {
        final result = await authService.login(
            email: ' admin@paperboost.com ',
            password: 'Admin123*',
        );

        expect(result.isSuccess, true);
    });

    test('Normalizar espacios en blanco en contraseña', () async {
        final result = await authService.login(
            email: 'admin@paperboost.com',
            password: ' Admin123* ',
        );

        expect(result.isSuccess, true);
    });

    test('Email case-insensitive', () async {
        final result = await authService.login(
            email: 'ADMIN@PAPERBOOST.COM',
            password: 'Admin123*',
        );

        expect(result.isSuccess, true);
    });
});

group('AuthService - Múltiples intentos de login', () {
    late AuthService authService;

    setUp(() {
        final userRepository = InMemoryUserRepository();
    });

```

```

    final sessionManager = SessionManager();
    authService = AuthService(
        userRepository: userRepository,
        sessionManager: sessionManager,
    );
});

test('Múltiples logins exitosos mantienen la sesión actualizada',
    () async {
        final result1 = await authService.login(
            email: 'admin@paperboost.com',
            password: 'Admin123*',
        );

        expect(result1.isSuccess, true);
        var currentUser = authService.currentUser;
        expect(currentUser!.id, 'USER-001');

        // Simular logout y nuevo login
        authService.logout();
        expect(authService.isAuthenticated, false);

        final result2 = await authService.login(
            email: 'admin@paperboost.com',
            password: 'Admin123*',
        );

        expect(result2.isSuccess, true);
        currentUser = authService.currentUser;
        expect(currentUser!.id, 'USER-001');
        expect(authService.isAuthenticated, true);
    });
});

group('AuthService - RQF-002: Registro de Usuario (Validaciones)', () {
    late AuthService authService;

    setUp(() {
        final userRepository = InMemoryUserRepository();
    });
});

```

```
final sessionManager = SessionManager();
authService = AuthService(
  userRepository: userRepository,
  sessionManager: sessionManager,
);
});

test('register rechaza campos vacíos', () async {
  final result = await authService.register(
    email: '',
    password: '',
  );
  expect(result.isSuccess, false);
  expect(result.message, contains('Todos los campos son obligatorios'));
});

test('register rechaza correo inválido', () async {
  final result = await authService.register(
    email: 'notanemail',
    password: 'Admin123*',
  );
  expect(result.isSuccess, false);
  expect(result.message, contains('formato del correo'));
});

test('register rechaza contraseña corta', () async {
  final result = await authService.register(
    email: 'test@paperboost.com',
    password: '12345',
  );
  expect(result.isSuccess, false);
  expect(result.message, contains('6 caracteres'));
});

test('register rechaza cuando ya existe usuario registrado', () async {
  final result = await authService.register(
    email: 'newuser@paperboost.com',
```

```

        password: 'NewUser123*',
    );
    expect(result.isSuccess, false);
    expect(result.message, contains('Ya existe un usuario
registrado')));
    });
});

group('AuthService - RQF-002: Registro de Usuario (Repositorio
limpio)', () {
    test('register crea cuenta exitosamente con repositorio limpio', ()
async {
        final userRepository = _CleanUserRepository();
        final sessionManager = SessionManager();
        final authService = AuthService(
            userRepository: userRepository,
            sessionManager: sessionManager,
        );

        final result = await authService.register(
            email: 'owner@paperboost.com',
            password: 'Owner123*',
        );

        expect(result.isSuccess, true);
        expect(result.data, isNotNull);
        expect(result.data!.email, 'owner@paperboost.com');
        expect(result.data!.passwordHash, isNotEmpty);
        expect(result.data!.passwordSalt, 'paperboost-owner-salt');
        expect(authService.isAuthenticated, true);
    });

    test('register maneja error del repositorio', () async {
        final userRepository = _ThrowingUserRepository();
        final sessionManager = SessionManager();
        final authService = AuthService(
            userRepository: userRepository,
            sessionManager: sessionManager,
        );
    });

```

```

        final result = await authService.register(
            email: 'owner@paperboost.com',
            password: 'Owner123*',
        );

        expect(result.isSuccess, false);
        expect(result.message, contains('No se pudo crear la cuenta'));
    });
}

```

```

class _CleanUserRepository implements UserRepository {
    final List<AppUser> _users = [];

    @override
    Future<AppUser?> findByEmail(String email) async {
        final normalized = email.trim().toLowerCase();
        for (final user in _users) {
            if (user.email.toLowerCase() == normalized) return user;
        }
        return null;
    }

    @override
    Future<bool> hasRegisteredUser() async => _users.isNotEmpty;

    @override
    Future<void> createUser(AppUser user) async {
        _users.add(user);
    }
}

```

```

class _ThrowingUserRepository implements UserRepository {
    @override
    Future<AppUser?> findByEmail(String email) async => null;

    @override
    Future<bool> hasRegisteredUser() async => false;
}

```

```
@override
Future<void> createUser(AppUser user) async {
  throw Exception('Simulated repository error');
}
}
```

Resultados Obtenidos

```
.../codigoSprints/paperboost ) flutter test test/logic/services/auth_service_test.dart
00:04 +23: All tests passed!

.../codigoSprints/paperboost ) █
```

RQF-002: Almacenamiento de credenciales de usuario

Como se Implementó

Archivo principal a testear: lib\data\security\password_hasher.dart

```
import 'dart:convert';

import 'package:crypto/crypto.dart';

class PasswordHasher {
  const PasswordHasher._();

  static String hash({
    required String password,
    required String salt,
  }) {
    final key = utf8.encode(salt);
    final passwordBytes = utf8.encode(password);

    final hmac = Hmac(sha256, key);
```

```

    final digest = hmac.convert(passwordBytes);

    return digest.toString();
}

static bool verify({
  required String password,
  required String salt,
  required String expectedHash,
}) {
  final calculatedHash = hash(
    password: password,
    salt: salt,
  );

  return calculatedHash == expectedHash;
}
}

```

Pruebas Realizadas

```

group('AuthService - RQF-002: Almacenamiento de Credenciales', () {
  late AuthService authService;

  setUp(() {
    final userRepository = InMemoryUserRepository();
    final sessionManager = SessionManager();
    authService = AuthService(
      userRepository: userRepository,
      sessionManager: sessionManager,
    );
  });

  test('Credenciales se almacenan de forma segura (hasheadas)', ()
  async {
    final result = await authService.login(
      email: 'admin@paperboost.com',
      password: 'Admin123*',
    );

    expect(result.isSuccess, true);
  });
});

```

```
    final user = result.data!;

    // Verificar que la contraseña no está en texto plano
    expect(user.passwordHash, isEmpty);
    expect(user.passwordHash, isNot('Admin123*'));
    expect(user.passwordSalt, isEmpty);
  });

  test('Usuario se almacena en sesión después del login', () async {
    expect(authService.currentUser, isNull);

    await authService.login(
      email: 'admin@paperboost.com',
      password: 'Admin123*',
    );

    expect(authService.currentUser, isNotNull);
    expect(authService.isAuthenticated, true);
  });

  test('Sesión se limpia al hacer logout', () async {
    await authService.login(
      email: 'admin@paperboost.com',
      password: 'Admin123*',
    );

    expect(authService.isAuthenticated, true);

    authService.logout();

    expect(authService.isAuthenticated, false);
    expect(authService.currentUser, isNull);
  });

  test('No se puede hacer logout sin sesión activa', () async {
    final result = authService.logout();

    expect(result.isSuccess, false);
  });
}
```

```

        expect(result.message, contains('No existe una sesión activa'));
    });

    test('isAuthenticated retorna true después del login', () async {
        expect(authService.isAuthenticated, false);

        await authService.login(
            email: 'admin@paperboost.com',
            password: 'Admin123*',
        );

        expect(authService.isAuthenticated, true);
    });

    test('currentUser retorna null cuando no está autenticado', () async {
        {
            expect(authService.currentUser, isNull);
        });
    });

    test('currentUser retorna el usuario autenticado', () async {
        await authService.login(
            email: 'admin@paperboost.com',
            password: 'Admin123*',
        );

        final user = authService.currentUser!;

        expect(user.email, 'admin@paperboost.com');
        expect(user.role, 'Administrador');
    });
});

```

Resultados Obtenidos

```

.../codigoSprints/paperboost ) flutter test test/logic/services/auth_service_test.dart --plain-name "AuthService - RQF-002: Almacenamiento de Credenciales"
00:02 +7: All tests passed!

```

RQF-007: Registrar producto manualmente

Como se implementó

Archivo principal a testear: lib\logic\services\product_service.dart

```
import '../data/models/product.dart';
import '../data/repositories/product_repository.dart';
import '../results/operation_result.dart';
import '../validators/product_validator.dart';

enum ProductSortOption {
  name,
  price,
  stock,
}

class ProductService {
  ProductService({
    required ProductRepository productRepository,
  }) : _productRepository = productRepository;

  final ProductRepository _productRepository;

  Future<OperationResult<Product>> registerProduct({
    required String sku,
    required String name,
    required double price,
    required int stock,
    required String category,
    required String location,
  }) async {
    final product = Product(
      id: _generateProductId(),
      sku: sku.trim().toUpperCase(),
      name: name.trim(),
      price: price,
      stock: stock,
      category: category.trim(),
      location: location.trim(),
    );

    final validationErrors =
      ProductValidator.validateProduct(product);

    if (validationErrors.isNotEmpty) {
      return OperationResult<Product>.failure(
        message: validationErrors.join('\n'),
      );
    }
  }
}
```

```

    }

    final duplicatedSku =
        await _productRepository.existsBySku(product.sku);

    if (duplicatedSku) {
        return OperationResult<Product>.failure(
            message:
                'Ya existe un producto registrado con el SKU ${product.sku}.',
        );
    }

    try {
        await _productRepository.save(product);

        return OperationResult<Product>.success(
            message: 'Producto registrado correctamente.',
            data: product,
        );
    } catch (error) {
        return OperationResult<Product>.failure(
            message: 'No se pudo registrar el producto: $error',
        );
    }
}

```

```

Future<OperationResult<List<Product>>> getProducts({
    bool includeInactive = false,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: includeInactive,
        );

        return OperationResult<List<Product>>.success(
            message: products.isEmpty
                ? 'No existen productos registrados.'
                : 'Productos consultados correctamente.',
            data: products,
        );
    } catch (error) {
        return OperationResult<List<Product>>.failure(
            message: 'No se pudo consultar el inventario: $error',
        );
    }
}

```

```

Future<OperationResult<Product>> getProductById(
    String id,
) async {

```

```

    if (id.trim().isEmpty) {
        return OperationResult<Product>.failure(
            message: 'El identificador del producto es obligatorio.',
        );
    }

    final product = await _productRepository.findById(
        id.trim(),
    );

    if (product == null) {
        return OperationResult<Product>.failure(
            message: 'Producto no encontrado.',
        );
    }

    return OperationResult<Product>.success(
        message: 'Producto encontrado.',
        data: product,
    );
}

Future<OperationResult<List<Product>>> searchProducts({
    String query = "",
    String? category,
    ProductStatus? status,
    ProductSortOption sortOption =
        ProductSortOption.name,
    bool ascending = true,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: true,
        );

        final normalizedQuery =
            query.trim().toLowerCase();

        final filteredProducts = products.where((product) {
            final matchesQuery =
                normalizedQuery.isEmpty ||
                product.name
                    .toLowerCase()
                    .contains(normalizedQuery) ||
                product.sku
                    .toLowerCase()
                    .contains(normalizedQuery);

            final matchesCategory =
                category == null ||

```

```

        category.trim().isEmpty ||
        product.category.toLowerCase() ==
            category.trim().toLowerCase());

    final matchesStatus =
        status == null || product.status == status;

    return matchesQuery &&
        matchesCategory &&
        matchesStatus;
}).toList();

_sortProducts(
    filteredProducts,
    sortOption: sortOption,
    ascending: ascending,
);

return OperationResult<List<Product>>.success(
    message: filteredProducts.isEmpty
        ? 'No se encontraron productos.'
        : 'Búsqueda realizada correctamente.',
    data: filteredProducts,
);
} catch (error) {
    return OperationResult<List<Product>>.failure(
        message: 'No se pudo realizar la búsqueda: $error',
    );
}
}

Future<OperationResult<Product>> updateProduct({
    required Product product,
}) async {
    final currentProduct =
        await _productRepository.findById(product.id);

    if (currentProduct == null) {
        return OperationResult<Product>.failure(
            message: 'El producto que desea editar no existe.',
        );
    }

    final normalizedProduct = product.copyWith(
        sku: product.sku.trim().toUpperCase(),
        name: product.name.trim(),
        category: product.category.trim(),
        location: product.location.trim(),
    );
}

```

```

final validationErrors =
    ProductValidator.validateProduct(
        normalizedProduct,
    );

if (validationErrors.isNotEmpty) {
    return OperationResult<Product>.failure(
        message: validationErrors.join('\n'),
    );
}

final duplicatedSku =
    await _productRepository.existsBySku(
        normalizedProduct.sku,
        excludingProductId: normalizedProduct.id,
    );

if (duplicatedSku) {
    return OperationResult<Product>.failure(
        message:
            'Ya existe otro producto con el SKU ${normalizedProduct.sku}.',
    );
}

try {
    await _productRepository.update(
        normalizedProduct,
    );

    return OperationResult<Product>.success(
        message: 'Producto actualizado correctamente.',
        data: normalizedProduct,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo actualizar el producto: $error',
    );
}
}

Future<OperationResult<Product>> deactivateProduct(
    String id,
) async {
    final product = await _productRepository.findById(id);

    if (product == null) {
        return OperationResult<Product>.failure(
            message:
                'El producto que desea dar de baja no existe.',
        );
    }
}

```

```

}

if (!product.isActive) {
    return OperationResult<Product>.failure(
        message: 'El producto ya se encuentra dado de baja.',
    );
}

try {
    await _productRepository.deactivate(id);

    final inactiveProduct = product.copyWith(
        status: ProductStatus.inactive,
    );

    return OperationResult<Product>.success(
        message: 'Producto dado de baja exitosamente.',
        data: inactiveProduct,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo dar de baja el producto: $error',
    );
}
}

void _sortProducts(
    List<Product> products, {
    required ProductSortOption sortOption,
    required bool ascending,
}) {
    products.sort((first, second) {
        int comparison;

        switch (sortOption) {
            case ProductSortOption.name:
                comparison = first.name
                    .toLowerCase()
                    .compareTo(second.name.toLowerCase());
                break;

            case ProductSortOption.price:
                comparison =
                    first.price.compareTo(second.price);
                break;

            case ProductSortOption.stock:
                comparison =
                    first.stock.compareTo(second.stock);
                break;
        }

        if (ascending) {
            return comparison;
        } else {
            return -comparison;
        }
    });
}

```

```

    }

    return ascending ? comparison : -comparison;
  });
}

String _generateProductId() {
  final timestamp =
    DateTime.now().microsecondsSinceEpoch;

  return 'PRODUCT-$timestamp';
}
}

```

Pruebas Realizadas

```

group('ProductService - RQF-007: Registrar Producto', () {
  late ProductService productService;
  late InMemoryProductRepository repository;

  setUp(() {
    repository = InMemoryProductRepository();
    productService = ProductService(
      productRepository: repository,
    );
  });

  test('registrar producto exitosamente', () async {
    final result = await productService.registerProduct(
      sku: 'SKU-001',
      name: 'Lápiz',
      price: 10.0,
      stock: 100,
      category: 'Papelería',
      location: 'A1',
    );

    expect(result.isSuccess, true);
    expect(result.data, isNotNull);
    expect(result.data!.sku, 'SKU-001');
  });
}

```

```
    expect(result.message, contains('Producto registrado
correctamente'));
  });

test('rechazar producto con validación fallida', () async {
  final result = await productService.registerProduct(
    sku: '',
    name: '',
    price: 0,
    stock: -1,
    category: '',
    location: '',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('El SKU es obligatorio'));
});

test('rechazar SKU duplicado', () async {
  await productService.registerProduct(
    sku: 'SKU-001',
    name: 'Lápiz',
    price: 10.0,
    stock: 100,
    category: 'Papelería',
    location: 'A1',
  );

  final result = await productService.registerProduct(
    sku: 'SKU-001',
    name: 'Otro',
    price: 20.0,
    stock: 50,
    category: 'Papelería',
    location: 'A2',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('Ya existe un producto registrado
```

```

con el SKU')));
});

test('normalizar SKU a mayúsculas', () async {
  final result = await productService.registerProduct(
    sku: ' sku-001 ',
    name: 'Producto',
    price: 10.0,
    stock: 10,
    category: 'Cat',
    location: 'A1',
  );

  expect(result.isSuccess, true);
  expect(result.data!.sku, 'SKU-001');
});
});

```

Resultados Obtenidos

```

../codigoSprints/paperboost ) flutter test test/logic/services/product_service_test.dart --plain-name "ProductService
00:03 +4: All tests passed!

```

RQF-008: Validar datos y SKU

Como se implementó

Archivo principal a testear: lib\logic\validators\product_validator.dart
import '../data/models/product.dart';

```

class ProductValidator {
  const ProductValidator._();

  static List<String> validateProduct(Product product) {
    final errors = <String>[];

    if (product.sku.trim().isEmpty) {

```

```
    errors.add('El SKU es obligatorio.');
```

```
  }
```

```
  if (product.name.trim().isEmpty) {
```

```
    errors.add('El nombre del producto es obligatorio.');
```

```
  }
```

```
  if (product.price <= 0) {
```

```
    errors.add('El precio debe ser mayor que cero.');
```

```
  }
```

```
  if (product.stock < 0) {
```

```
    errors.add('El stock no puede ser negativo.');
```

```
  }
```

```
  if (product.category.trim().isEmpty) {
```

```
    errors.add('La categoría es obligatoria.');
```

```
  }
```

```
  if (product.location.trim().isEmpty) {
```

```
    errors.add('La ubicación es obligatoria.');
```

```
  }
```

```
  return errors;
```

```
}
```

```
}
```

Pruebas Realizadas

```
test('rechazar producto con validación fallida', () async {
```

```
  final result = await productService.registerProduct(  
    sku: '',  
    name: '',  
    price: 0,  
    stock: -1,  
    category: '',  
    location: '',  
  );
```

```
  expect(result.isSuccess, false);  
  expect(result.message, contains('El SKU es obligatorio'));
```

```
});
```

```

test('rechazar SKU duplicado', () async {
  await productService.registerProduct(
    sku: 'SKU-001',
    name: 'Lápiz',
    price: 10.0,
    stock: 100,
    category: 'Papelería',
    location: 'A1',
  );

  final result = await productService.registerProduct(
    sku: 'SKU-001',
    name: 'Otro',
    price: 20.0,
    stock: 50,
    category: 'Papelería',
    location: 'A2',
  );

  expect(result.isSuccess, false);
  expect(result.message, contains('Ya existe un producto registrado
con el SKU'));
});

test('normalizar SKU a mayúsculas', () async {
  final result = await productService.registerProduct(
    sku: ' sku-001 ',
    name: 'Producto',
    price: 10.0,
    stock: 10,
    category: 'Cat',
    location: 'A1',
  );

  expect(result.isSuccess, true);
  expect(result.data!.sku, 'SKU-001');
});
});

```

Resultados Obtenidos

```
../codigoSprints/paperboost ) flutter test test/logic/services/product_service_test.dart --plain-name "ProductService  
00:03 +4: All tests passed!
```

RQF-009: Generar nota de venta

Como se implementó

Archivo principal a testear: lib\logic\services\sale_service.dart

```
import '../data/models/sale.dart';  
import '../data/models/sale_item.dart';  
import '../data/repositories/product_repository.dart';  
import '../data/repositories/sale_repository.dart';  
import '../observers/stock_change_notifier.dart';  
import '../results/operation_result.dart';  
import '../validators/sale_validator.dart';  
  
class SaleService {  
  SaleService({  
    required SaleRepository saleRepository,  
    required ProductRepository productRepository,  
    StockChangeNotifier? stockChangeNotifier,  
  }) : _saleRepository = saleRepository,  
       _productRepository = productRepository,  
       _stockChangeNotifier =  
         stockChangeNotifier ?? StockChangeNotifier.instance;  
  
  static const double _taxRate = 0.15; // 15% IVA (ajustar según país)  
  static int _saleIdCounter = 0;  
  
  final SaleRepository _saleRepository;  
  final ProductRepository _productRepository;  
  final StockChangeNotifier _stockChangeNotifier;  
  
  Future<OperationResult<Sale>> createSale({  
    required List<SaleItem> items,  
    required PaymentMethod paymentMethod,  
    String observations = "",  
    String customerName = "",  
    String customerEmail = "",  
  })
```

```

}) async {
  // Validar items
  final itemErrors = SaleValidator.validateSaleItems(items);
  if (itemErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: itemErrors.join("\n"),
    );
  }

  // Validar disponibilidad de stock
  final stockErrors = SaleValidator.validateStockForAllItems(items);
  if (stockErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: stockErrors.join("\n"),
    );
  }

  // Validar información del cliente (opcional)
  final customerErrors = SaleValidator.validateCustomerInfo(
    customerName.isEmpty ? null : customerName,
    customerEmail.isEmpty ? null : customerEmail,
  );
  if (customerErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: customerErrors.join("\n"),
    );
  }

  try {
    // Calcular totales
    final subtotal = items.fold<double>(
      0,
      (sum, item) => sum + item.subtotal,
    );

    final taxAmount = subtotal * _taxRate;
    final total = subtotal + taxAmount;

    // Generar número de venta
    final nextNumber = await _saleRepository.getNextSaleNumber();
    final saleNumber = _formatSaleNumber(nextNumber);

    // Crear venta
    final sale = Sale(
      id: _generateSaleId(),
      saleNumber: saleNumber,
      items: items,
      subtotal: subtotal,
      taxAmount: taxAmount,
      total: total,
    );
  } catch (e) {
    // Manejar errores de creación de venta
  }
}

```

```

        paymentMethod: paymentMethod,
        status: SaleStatus.completed,
        createdAt: DateTime.now(),
        observations: observations.trim(),
        customerName: customerName.trim(),
        customerEmail: customerEmail.trim(),
    );

    // Guardar venta
    await _saleRepository.save(sale);

    // Descontar stock y notificar
    await _decrementProductsStock(items);

    return OperationResult<Sale>.success(
        message: 'Nota de venta creada correctamente.',
        data: sale,
    );
} catch (error) {
    return OperationResult<Sale>.failure(
        message: 'No se pudo crear la nota de venta: $error',
    );
}
}

Future<OperationResult<Sale>> completeSale(String saleId) async {
    try {
        final sale = await _saleRepository.findById(saleId);

        if (sale == null) {
            return OperationResult<Sale>.failure(
                message: 'No se encontró la venta.',
            );
        }

        if (sale.isCompleted) {
            return OperationResult<Sale>.success(
                message: 'La venta ya estaba completada.',
                data: sale,
            );
        }

        if (!sale.isPending) {
            return OperationResult<Sale>.failure(
                message: 'Solo se pueden completar ventas pendientes.',
            );
        }

        final completedSale =
            sale.copyWith(status: SaleStatus.completed);
    }
}

```

```

        await _saleRepository.update(completedSale);

        return OperationResult<Sale>.success(
            message: 'Venta completada correctamente.',
            data: completedSale,
        );
    } catch (error) {
        return OperationResult<Sale>.failure(
            message: 'No se pudo completar la venta: $error',
        );
    }
}

Future<OperationResult<void>> cancelSale(String saleId) async {
    try {
        final sale = await _saleRepository.findById(saleId);

        if (sale == null) {
            return OperationResult<void>.failure(
                message: 'No se encontró la venta.',
            );
        }

        if (sale.isCancelled) {
            return OperationResult<void>.failure(
                message: 'La venta ya está cancelada.',
            );
        }

        // Revertir stock
        await _restoreProductsStock(sale.items);

        // Cancelar venta
        await _saleRepository.cancel(saleId);

        return OperationResult<void>.success(
            message: 'Venta cancelada correctamente.',
        );
    } catch (error) {
        return OperationResult<void>.failure(
            message: 'No se pudo cancelar la venta: $error',
        );
    }
}

Future<OperationResult<List<Sale>>> getSales({
    bool includeCancelled = false,
}) async {
    try {

```

```

    final sales = await _saleRepository.findAll(
      includeCompleted: true,
      includeCancelled: includeCancelled,
    );

    return OperationResult<List<Sale>>.success(
      message: sales.isEmpty
        ? 'No hay ventas registradas.'
        : 'Ventas consultadas correctamente.',
      data: sales,
    );
  } catch (error) {
    return OperationResult<List<Sale>>.failure(
      message: 'No se pudo consultar las ventas: $error',
    );
  }
}

Future<OperationResult<Sale>> getSaleById(String saleId) async {
  try {
    final sale = await _saleRepository.findById(saleId);

    if (sale == null) {
      return OperationResult<Sale>.failure(
        message: 'No se encontró la venta.',
      );
    }

    return OperationResult<Sale>.success(
      message: 'Venta consultada correctamente.',
      data: sale,
    );
  } catch (error) {
    return OperationResult<Sale>.failure(
      message: 'No se pudo consultar la venta: $error',
    );
  }
}

Future<OperationResult<List<Sale>>> searchSales({
  String query = "",
  SaleStatus? status,
}) async {
  try {
    final sales = await _saleRepository.findAll(
      includeCompleted: true,
      includeCancelled: true,
    );

    final normalizedQuery = query.trim().toLowerCase();

```

```

final filteredSales = sales.where((sale) {
    final matchesQuery = normalizedQuery.isEmpty ||
        sale.saleNumber.toLowerCase().contains(normalizedQuery) ||
        sale.customerName.toLowerCase().contains(normalizedQuery) ||
        sale.customerEmail.toLowerCase().contains(normalizedQuery);

    final matchesStatus = status == null || sale.status == status;

    return matchesQuery && matchesStatus;
}).toList();

return OperationResult<List<Sale>>.success(
    message: filteredSales.isEmpty
        ? 'No se encontraron ventas.'
        : 'Ventas consultadas correctamente.',
    data: filteredSales,
);
} catch (error) {
    return OperationResult<List<Sale>>.failure(
        message: 'No se pudo realizar la búsqueda: $error',
    );
}
}

Future<void> _decrementProductsStock(List<SaleItem> items) async {
    for (final item in items) {
        final product = await _productRepository.findById(
            item.product.id,
        );

        if (product != null) {
            final oldStock = product.stock;
            final newStock = product.stock - item.quantity;
            final updatedProduct = product.copyWith(stock: newStock);

            await _productRepository.update(updatedProduct);

            // Notificar cambio de stock
            _stockChangeNotifier.notifyStockChanged(
                product,
                oldStock,
                newStock,
            );

            // Si el stock llega a 0, notificar que no hay disponibilidad
            if (newStock == 0) {
                _stockChangeNotifier.notifyStockUnavailable(updatedProduct);
            }
        }
    }
}

```

```

    }
  }

Future<void> _restoreProductsStock(List<SaleItem> items) async {
  for (final item in items) {
    final product = await _productRepository.findById(
      item.product.id,
    );

    if (product != null) {
      final oldStock = product.stock;
      final newStock = product.stock + item.quantity;
      final updatedProduct = product.copyWith(stock: newStock);

      await _productRepository.update(updatedProduct);

      _stockChangeNotifier.notifyStockChanged(
        product,
        oldStock,
        newStock,
      );
    }
  }
}

String _formatSaleNumber(int number) {
  return 'VTA-${number.toString().padLeft(6, '0')}';
}

String _generateSaleId() {
  _saleIdCounter++;
  return 'SALE-${_saleIdCounter.toString().padLeft(4, '0')}';
}
}

```

Pruebas Realizadas

[PEGAR AQUÍ EL CÃ“DIGO DE LA PRUEBA UNITARIA / WIDGET / INTEGRACIÃ“N]

Resultados Obtenidos

[COMPLETAR RESULTADOS, ESTADO, OBSERVACIONES Y EVIDENCIA]

RQF-010: Validar stock de la venta

CÃ“mo se ImplementÃ³

Archivo principal a testear: lib\logic\validators\sale_validator.dart

import '../data/models/sale_item.dart';

```

class SaleValidator {
  const SaleValidator._();

  static List<String> validateSaleItems(List<SaleItem> items) {
    final errors = <String>[];

```

```

    if (items.isEmpty) {
        errors.add('Debe agregar al menos un producto a la venta.');
```

```

        return errors;
    }

    for (int i = 0; i < items.length; i++) {
        final item = items[i];
        final itemErrors = _validateSaleItem(item, i + 1);
        errors.addAll(itemErrors);
    }

    return errors;
}

static List<String> _validateSaleItem(
    SaleItem item,
    int itemIndex,
) {
    final errors = <String>[];

    if (item.quantity <= 0) {
        errors.add(
            'Producto $itemIndex: La cantidad debe ser mayor que cero.',
        );
    }

    if (item.unitPrice < 0) {
        errors.add(
            'Producto $itemIndex: El precio unitario no puede ser negativo.',
        );
    }

    return errors;
}

static List<String> validateStockAvailability(
    SaleItem item,
) {
    final errors = <String>[];

    if (item.quantity > item.product.stock) {
        errors.add(
            'Stock insuficiente para "${item.product.name} ". '
            'Disponible: ${item.product.stock}, Solicitado: ${item.quantity}',
        );
    }

    return errors;
}

```

```

static List<String> validateStockForAllItems(
    List<SaleItem> items,
) {
    final errors = <String>[];

    for (final item in items) {
        final stockErrors = validateStockAvailability(item);
        errors.addAll(stockErrors);
    }

    return errors;
}

static List<String> validatePaymentMethod(
    String? paymentMethodName,
) {
    final errors = <String>[];

    if (paymentMethodName == null || paymentMethodName.trim().isEmpty) {
        errors.add('Debe seleccionar un método de pago.');
```

```
    return emailExpression.hasMatch(email);  
  }  
}
```

Pruebas Realizadas

```
group('SaleService - Crear Venta', () {  
  late SaleService saleService;  
  late InMemorySaleRepository saleRepository;  
  late InMemoryProductRepository productRepository;  
  late Product testProduct;  
  
  setUp(() async {  
    saleRepository = InMemorySaleRepository();  
    productRepository = InMemoryProductRepository();  
  
    testProduct = const Product(  
      id: 'PROD-001',  
      sku: 'SKU-001',  
      name: 'Product Test',  
      price: 100.0,  
      stock: 50,  
      category: 'Test',  
      location: 'A1',  
    );  
  
    await productRepository.save(testProduct);  
  
    saleService = SaleService(  
      saleRepository: saleRepository,  
      productRepository: productRepository,  
    );  
  });  
  
  test('Crear venta con items válidos', () async {  
    final item = SaleItem(  
      id: 'ITEM-001',  

```

```
        product: testProduct,
        quantity: 5,
        unitPrice: 100.0,
    );

    final result = await saleService.createSale(
        items: [item],
        paymentMethod: PaymentMethod.cash,
    );

    expect(result.isSuccess, true);
    expect(result.data, isNotNull);
    expect(result.data!.total, 575.0); // 500 + 75 IVA
});

test('Fallar al crear venta sin items', () async {
    final result = await saleService.createSale(
        items: [],
        paymentMethod: PaymentMethod.cash,
    );

    expect(result.isSuccess, false);
    expect(result.message, contains('al menos un producto'));
});

test('Fallar al crear venta con stock insuficiente', () async {
    final item = SaleItem(
        id: 'ITEM-001',
        product: testProduct,
        quantity: 100, // Mayor que stock disponible (50)
        unitPrice: 100.0,
    );

    final result = await saleService.createSale(
        items: [item],
        paymentMethod: PaymentMethod.cash,
    );

    expect(result.isSuccess, false);
```

```
        expect(result.message, contains('Stock insuficiente'));
    });

    test('Descontar stock automáticamente después de crear venta', ()
async {
        final item = SaleItem(
            id: 'ITEM-001',
            product: testProduct,
            quantity: 10,
            unitPrice: 100.0,
        );

        await saleService.createSale(
            items: [item],
            paymentMethod: PaymentMethod.cash,
        );

        final updatedProduct = await productRepository.findById('PROD-
001');

        expect(updatedProduct!.stock, 40); // 50 - 10
    });

    test('Generar número de venta único', () async {
        final item = SaleItem(
            id: 'ITEM-001',
            product: testProduct,
            quantity: 1,
            unitPrice: 100.0,
        );

        final result1 = await saleService.createSale(
            items: [item],
            paymentMethod: PaymentMethod.cash,
        );

        final result2 = await saleService.createSale(
            items: [item],
            paymentMethod: PaymentMethod.cash,
```

```

    );

    expect(result1.data!.saleNumber, isNot(result2.data!.saleNumber));
  });

  test('Calcular IVA correctamente (15%)', () async {
    final item = SaleItem(
      id: 'ITEM-001',
      product: testProduct,
      quantity: 1,
      unitPrice: 100.0,
    );

    final result = await saleService.createSale(
      items: [item],
      paymentMethod: PaymentMethod.cash,
    );

    expect(result.data!.subtotal, 100.0);
    expect(result.data!.taxAmount, 15.0);
    expect(result.data!.total, 115.0);
  });
});

```

Resultados Obtenidos

```

.../codigoSprints/paperboost x flutter test test/logic/services/sale_service_test.dart --plain-name "SaleService - Crear Venta"
00:02 +6: All tests passed!

```

RQF-011: Consultar inventario

Como se implementó

Archivo principal a testear: lib\logic\services\product_service.dart

```

import '../data/models/product.dart';
import '../data/repositories/product_repository.dart';
import '../results/operation_result.dart';
import '../validators/product_validator.dart';

enum ProductSortOption {
  name,
  price,
  stock,
}

class ProductService {
  ProductService({
    required ProductRepository productRepository,
  }) : _productRepository = productRepository;

  final ProductRepository _productRepository;

  Future<OperationResult<Product>> registerProduct({
    required String sku,
    required String name,
    required double price,
    required int stock,
    required String category,
    required String location,
  }) async {
    final product = Product(
      id: _generateProductId(),
      sku: sku.trim().toUpperCase(),
      name: name.trim(),
      price: price,
      stock: stock,
      category: category.trim(),
      location: location.trim(),
    );

    final validationErrors =
      ProductValidator.validateProduct(product);

    if (validationErrors.isNotEmpty) {
      return OperationResult<Product>.failure(
        message: validationErrors.join('\n'),
      );
    }

    final duplicatedSku =
      await _productRepository.existsBySku(product.sku);

    if (duplicatedSku) {
      return OperationResult<Product>.failure(

```

```

        message:
            'Ya existe un producto registrado con el SKU ${product.sku}.',
    );
}

try {
    await _productRepository.save(product);

    return OperationResult<Product>.success(
        message: 'Producto registrado correctamente.',
        data: product,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo registrar el producto: $error',
    );
}
}

```

```

Future<OperationResult<List<Product>>> getProducts({
    bool includeInactive = false,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: includeInactive,
        );

        return OperationResult<List<Product>>.success(
            message: products.isEmpty
                ? 'No existen productos registrados.'
                : 'Productos consultados correctamente.',
            data: products,
        );
    } catch (error) {
        return OperationResult<List<Product>>.failure(
            message: 'No se pudo consultar el inventario: $error',
        );
    }
}

```

```

Future<OperationResult<Product>> getProductById(
    String id,
) async {
    if (id.trim().isEmpty) {
        return OperationResult<Product>.failure(
            message: 'El identificador del producto es obligatorio.',
        );
    }
}

```

```

final product = await _productRepository.findById(

```

```

        id.trim(),
    );

    if (product == null) {
        return OperationResult<Product>.failure(
            message: 'Producto no encontrado.',
        );
    }

    return OperationResult<Product>.success(
        message: 'Producto encontrado.',
        data: product,
    );
}

Future<OperationResult<List<Product>>> searchProducts({
    String query = "",
    String? category,
    ProductStatus? status,
    ProductSortOption sortOption =
        ProductSortOption.name,
    bool ascending = true,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: true,
        );

        final normalizedQuery =
            query.trim().toLowerCase();

        final filteredProducts = products.where((product) {
            final matchesQuery =
                normalizedQuery.isEmpty ||
                product.name
                    .toLowerCase()
                    .contains(normalizedQuery) ||
                product.sku
                    .toLowerCase()
                    .contains(normalizedQuery);

            final matchesCategory =
                category == null ||
                category.trim().isEmpty ||
                product.category.toLowerCase() ==
                    category.trim().toLowerCase();

            final matchesStatus =
                status == null || product.status == status;

```

```

        return matchesQuery &&
            matchesCategory &&
            matchesStatus;
    }).toList();

    _sortProducts(
        filteredProducts,
        sortOption: sortOption,
        ascending: ascending,
    );

    return OperationResult<List<Product>>.success(
        message: filteredProducts.isEmpty
            ? 'No se encontraron productos.'
            : 'Búsqueda realizada correctamente.',
        data: filteredProducts,
    );
} catch (error) {
    return OperationResult<List<Product>>.failure(
        message: 'No se pudo realizar la búsqueda: $error',
    );
}
}

```

```

Future<OperationResult<Product>> updateProduct({
    required Product product,
}) async {
    final currentProduct =
        await _productRepository.findById(product.id);

    if (currentProduct == null) {
        return OperationResult<Product>.failure(
            message: 'El producto que desea editar no existe.',
        );
    }
}

```

```

final normalizedProduct = product.copyWith(
    sku: product.sku.trim().toUpperCase(),
    name: product.name.trim(),
    category: product.category.trim(),
    location: product.location.trim(),
);

```

```

final validationErrors =
    ProductValidator.validateProduct(
        normalizedProduct,
    );

```

```

if (validationErrors.isNotEmpty) {
    return OperationResult<Product>.failure(

```

```

        message: validationErrors.join('\n'),
    );
}

final duplicatedSku =
    await _productRepository.existsBySku(
        normalizedProduct.sku,
        excludingProductId: normalizedProduct.id,
    );

if (duplicatedSku) {
    return OperationResult<Product>.failure(
        message:
            'Ya existe otro producto con el SKU ${normalizedProduct.sku}.',
    );
}

try {
    await _productRepository.update(
        normalizedProduct,
    );

    return OperationResult<Product>.success(
        message: 'Producto actualizado correctamente.',
        data: normalizedProduct,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo actualizar el producto: $error',
    );
}
}

Future<OperationResult<Product>> deactivateProduct(
    String id,
) async {
    final product = await _productRepository.findById(id);

    if (product == null) {
        return OperationResult<Product>.failure(
            message:
                'El producto que desea dar de baja no existe.',
        );
    }

    if (!product.isActive) {
        return OperationResult<Product>.failure(
            message: 'El producto ya se encuentra dado de baja.',
        );
    }
}

```

```

try {
    await _productRepository.deactivate(id);

    final inactiveProduct = product.copyWith(
        status: ProductStatus.inactive,
    );

    return OperationResult<Product>.success(
        message: 'Producto dado de baja exitosamente.',
        data: inactiveProduct,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo dar de baja el producto: $error',
    );
}
}

```

```

void _sortProducts(
    List<Product> products, {
    required ProductSortOption sortOption,
    required bool ascending,
}) {
    products.sort((first, second) {
        int comparison;

        switch (sortOption) {
            case ProductSortOption.name:
                comparison = first.name
                    .toLowerCase()
                    .compareTo(second.name.toLowerCase());
                break;

            case ProductSortOption.price:
                comparison =
                    first.price.compareTo(second.price);
                break;

            case ProductSortOption.stock:
                comparison =
                    first.stock.compareTo(second.stock);
                break;
        }

        return ascending ? comparison : -comparison;
    });
}

```

```

String _generateProductId() {

```

```
final timestamp =  
    DateTime.now().microsecondsSinceEpoch;  
  
return 'PRODUCT-$timestamp';  
}  
}
```

Pruebas Realizadas

```
group('ProductService - RQF-011: Consultar Inventario', () {  
    late ProductService productService;  
    late InMemoryProductRepository repository;  
  
    setUp(() async {  
        repository = InMemoryProductRepository();  
        productService = ProductService(  
            productRepository: repository,  
        );  
  
        await productService.registerProduct(  
            sku: 'SKU-001',  
            name: 'Lápiz',  
            price: 10.0,  
            stock: 100,  
            category: 'Papelería',  
            location: 'A1',  
        );  
  
        await productService.registerProduct(  
            sku: 'SKU-002',  
            name: 'Cuaderno',  
            price: 25.0,  
            stock: 50,  
            category: 'Papelería',  
            location: 'B1',  
        );  
    });  
  
    test('getProducts retorna todos los productos activos', () async {
```

```
    final result = await productService.getProducts();
    expect(result.isSuccess, true);
    expect(result.data, hasLength(2));
  });

test('getProducts retorna mensaje si no hay productos', () async {
  final emptyService = ProductService(
    productRepository: InMemoryProductRepository(),
  );
  final result = await emptyService.getProducts();
  expect(result.isSuccess, true);
  expect(result.message, contains('No existen productos
registrados'));
});

test('getProducts incluye inactivos si se solicita', () async {
  final allProducts = await productService.getProducts();
  final productId = allProducts.data!.first.id;

  await productService.deactivateProduct(productId);

  final activeOnly = await productService.getProducts();
  expect(activeOnly.data, hasLength(1));

  final includeInactive = await productService.getProducts(
    includeInactive: true,
  );
  expect(includeInactive.data, hasLength(2));
});

test('getProductById encuentra producto', () async {
  final savedResult = await productService.registerProduct(
    sku: 'SKU-003',
    name: 'Producto',
    price: 30.0,
    stock: 20,
    category: 'Cat',
    location: 'C1',
  );
});
```

```

    final result = await productService.getProductById(
      savedResult.data!.id,
    );
    expect(result.isSuccess, true);
    expect(result.data!.sku, 'SKU-003');
  });

  test('getProductById rechaza ID vacío', () async {
    final result = await productService.getProductById('');
    expect(result.isSuccess, false);
    expect(result.message, contains('obligatorio'));
  });

  test('getProductById retorna failure para ID inexistente', () async {
    final result = await productService.getProductById('NO-EXISTE');
    expect(result.isSuccess, false);
    expect(result.message, contains('Producto no encontrado'));
  });
});

```

Resultados Obtenidos

```

../codigoSprints/paperboost ) flutter test test/logic/services/product_service_test.dart --plain-name "ProductService - RQF-011: Consultar Inventario"
00:03 +6: All tests passed!

```

RQF-012: Buscar y filtrar inventario

Como se implementó

Archivo principal a testear: lib\logic\services\product_service.dart

```

import '../data/models/product.dart';
import '../data/repositories/product_repository.dart';
import '../results/operation_result.dart';
import '../validators/product_validator.dart';

enum ProductSortOption {

```

```

    name,
    price,
    stock,
}

class ProductService {
    ProductService({
        required ProductRepository productRepository,
    }) : _productRepository = productRepository;

    final ProductRepository _productRepository;

    Future<OperationResult<Product>> registerProduct({
        required String sku,
        required String name,
        required double price,
        required int stock,
        required String category,
        required String location,
    }) async {
        final product = Product(
            id: _generateProductId(),
            sku: sku.trim().toUpperCase(),
            name: name.trim(),
            price: price,
            stock: stock,
            category: category.trim(),
            location: location.trim(),
        );

        final validationErrors =
            ProductValidator.validateProduct(product);

        if (validationErrors.isNotEmpty) {
            return OperationResult<Product>.failure(
                message: validationErrors.join('\n'),
            );
        }

        final duplicatedSku =
            await _productRepository.existsBySku(product.sku);

        if (duplicatedSku) {
            return OperationResult<Product>.failure(
                message:
                    'Ya existe un producto registrado con el SKU ${product.sku}.',
            );
        }

        try {

```

```

    await _productRepository.save(product);

    return OperationResult<Product>.success(
        message: 'Producto registrado correctamente.',
        data: product,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo registrar el producto: $error',
    );
}
}

Future<OperationResult<List<Product>>> getProducts({
    bool includeInactive = false,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: includeInactive,
        );

        return OperationResult<List<Product>>.success(
            message: products.isEmpty
                ? 'No existen productos registrados.'
                : 'Productos consultados correctamente.',
            data: products,
        );
    } catch (error) {
        return OperationResult<List<Product>>.failure(
            message: 'No se pudo consultar el inventario: $error',
        );
    }
}

Future<OperationResult<Product>> getProductById(
    String id,
) async {
    if (id.trim().isEmpty) {
        return OperationResult<Product>.failure(
            message: 'El identificador del producto es obligatorio.',
        );
    }

    final product = await _productRepository.findById(
        id.trim(),
    );

    if (product == null) {
        return OperationResult<Product>.failure(
            message: 'Producto no encontrado.',
        );
    }
}

```

```

    );
}

return OperationResult<Product>.success(
    message: 'Producto encontrado.',
    data: product,
);
}

Future<OperationResult<List<Product>>> searchProducts({
    String query = "",
    String? category,
    ProductStatus? status,
    ProductSortOption sortOption =
        ProductSortOption.name,
    bool ascending = true,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: true,
        );

        final normalizedQuery =
            query.trim().toLowerCase();

        final filteredProducts = products.where((product) {
            final matchesQuery =
                normalizedQuery.isEmpty ||
                product.name
                    .toLowerCase()
                    .contains(normalizedQuery) ||
                product.sku
                    .toLowerCase()
                    .contains(normalizedQuery);

            final matchesCategory =
                category == null ||
                category.trim().isEmpty ||
                product.category.toLowerCase() ==
                    category.trim().toLowerCase();

            final matchesStatus =
                status == null || product.status == status;

            return matchesQuery &&
                matchesCategory &&
                matchesStatus;
        }).toList();

        _sortProducts(

```

```

        filteredProducts,
        sortOption: sortOption,
        ascending: ascending,
    );

    return OperationResult<List<Product>>.success(
        message: filteredProducts.isEmpty
            ? 'No se encontraron productos.'
            : 'Búsqueda realizada correctamente.',
        data: filteredProducts,
    );
} catch (error) {
    return OperationResult<List<Product>>.failure(
        message: 'No se pudo realizar la búsqueda: $error',
    );
}
}

Future<OperationResult<Product>> updateProduct({
    required Product product,
}) async {
    final currentProduct =
        await _productRepository.findById(product.id);

    if (currentProduct == null) {
        return OperationResult<Product>.failure(
            message: 'El producto que desea editar no existe.',
        );
    }

    final normalizedProduct = product.copyWith(
        sku: product.sku.trim().toUpperCase(),
        name: product.name.trim(),
        category: product.category.trim(),
        location: product.location.trim(),
    );

    final validationErrors =
        ProductValidator.validateProduct(
            normalizedProduct,
        );

    if (validationErrors.isNotEmpty) {
        return OperationResult<Product>.failure(
            message: validationErrors.join('\n'),
        );
    }

    final duplicatedSku =
        await _productRepository.existsBySku(

```

```

        normalizedProduct.sku,
        excludingProductId: normalizedProduct.id,
    );

    if (duplicatedSku) {
        return OperationResult<Product>.failure(
            message:
                'Ya existe otro producto con el SKU ${normalizedProduct.sku}.',
        );
    }

    try {
        await _productRepository.update(
            normalizedProduct,
        );

        return OperationResult<Product>.success(
            message: 'Producto actualizado correctamente.',
            data: normalizedProduct,
        );
    } catch (error) {
        return OperationResult<Product>.failure(
            message: 'No se pudo actualizar el producto: $error',
        );
    }
}

Future<OperationResult<Product>> deactivateProduct(
    String id,
) async {
    final product = await _productRepository.findById(id);

    if (product == null) {
        return OperationResult<Product>.failure(
            message:
                'El producto que desea dar de baja no existe.',
        );
    }

    if (!product.isActive) {
        return OperationResult<Product>.failure(
            message: 'El producto ya se encuentra dado de baja.',
        );
    }

    try {
        await _productRepository.deactivate(id);

        final inactiveProduct = product.copyWith(
            status: ProductStatus.inactive,

```

```

);

return OperationResult<Product>.success(
    message: 'Producto dado de baja exitosamente.',
    data: inactiveProduct,
);
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo dar de baja el producto: $error',
    );
}
}

void _sortProducts(
    List<Product> products, {
    required ProductSortOption sortOption,
    required bool ascending,
}) {
    products.sort((first, second) {
        int comparison;

        switch (sortOption) {
            case ProductSortOption.name:
                comparison = first.name
                    .toLowerCase()
                    .compareTo(second.name.toLowerCase());
                break;

            case ProductSortOption.price:
                comparison =
                    first.price.compareTo(second.price);
                break;

            case ProductSortOption.stock:
                comparison =
                    first.stock.compareTo(second.stock);
                break;
        }

        return ascending ? comparison : -comparison;
    });
}

String _generateProductId() {
    final timestamp =
        DateTime.now().microsecondsSinceEpoch;

    return 'PRODUCT-$timestamp';
}
}

```

Pruebas Realizadas

```
group('ProductService - RQF-012: Buscar Productos', () {
    late ProductService productService;

    setUp(() async {
        final repo = InMemoryProductRepository();
        productService = ProductService(productRepository: repo);

        await productService.registerProduct(
            sku: 'LAP-001',
            name: 'Lápiz Grafito',
            price: 10.0,
            stock: 100,
            category: 'Papelería',
            location: 'A1',
        );
        await productService.registerProduct(
            sku: 'CUA-001',
            name: 'Cuaderno Profesional',
            price: 35.0,
            stock: 50,
            category: 'Papelería',
            location: 'B1',
        );
        await productService.registerProduct(
            sku: 'RES-001',
            name: 'Resma Papel',
            price: 80.0,
            stock: 20,
            category: 'Papel',
            location: 'C1',
        );
    });

    test('buscar por nombre', () async {
        final result = await productService.searchProducts(query: 'Lápiz');
```

```
    expect(result.isSuccess, true);
    expect(result.data, hasLength(1));
    expect(result.data!.first.name, 'Lápiz Grafito');
  });

  test('buscar por SKU', () async {
    final result = await productService.searchProducts(query: 'CUA');
    expect(result.isSuccess, true);
    expect(result.data, hasLength(1));
  });

  test('buscar sin query retorna todos', () async {
    final result = await productService.searchProducts();
    expect(result.data, hasLength(3));
  });

  test('buscar sin coincidencias', () async {
    final result = await productService.searchProducts(query: 'zzzzz');
    expect(result.isSuccess, true);
    expect(result.data, isEmpty);
    expect(result.message, contains('No se encontraron productos'));
  });

  test('filtrar por categoría', () async {
    final result = await productService.searchProducts(category:
'Papel');
    expect(result.data, hasLength(1));
  });

  test('ordenar por precio ascendente', () async {
    final result = await productService.searchProducts(
      sortOption: ProductSortOption.price,
      ascending: true,
    );
    expect(result.data![0].price, 10.0);
    expect(result.data![2].price, 80.0);
  });

  test('ordenar por stock descendente', () async {
```

```

    final result = await productService.searchProducts(
      sortOption: ProductSortOption.stock,
      ascending: false,
    );
    expect(result.data![0].stock, 100);
  });

  test('filtrar por estado activo', () async {
    final result = await productService.searchProducts(
      status: ProductStatus.active,
    );
    expect(result.isSuccess, true);
    expect(result.data, hasLength(3));
  });
});

```

Resultados Obtenidos

```

.../codigoSprints/paperboost ) flutter test test/logic/services/product_service_test.dart --plain-name "ProductService - RQF-012: Buscar Productos"
00:03 +8: All tests passed!

```

RQF-013: Editar información del producto

Como se implementó

Archivo principal a testear: lib\logic\services\product_service.dart

```

import '../data/models/product.dart';
import '../data/repositories/product_repository.dart';
import '../results/operation_result.dart';
import '../validators/product_validator.dart';

enum ProductSortOption {
  name,
  price,
  stock,
}

```

```

class ProductService {
    ProductService({
        required ProductRepository productRepository,
    }) : _productRepository = productRepository;

    final ProductRepository _productRepository;

    Future<OperationResult<Product>> registerProduct({
        required String sku,
        required String name,
        required double price,
        required int stock,
        required String category,
        required String location,
    }) async {
        final product = Product(
            id: _generateProductId(),
            sku: sku.trim().toUpperCase(),
            name: name.trim(),
            price: price,
            stock: stock,
            category: category.trim(),
            location: location.trim(),
        );

        final validationErrors =
            ProductValidator.validateProduct(product);

        if (validationErrors.isNotEmpty) {
            return OperationResult<Product>.failure(
                message: validationErrors.join('\n'),
            );
        }

        final duplicatedSku =
            await _productRepository.existsBySku(product.sku);

        if (duplicatedSku) {
            return OperationResult<Product>.failure(
                message:
                    'Ya existe un producto registrado con el SKU ${product.sku}.',
            );
        }

        try {
            await _productRepository.save(product);

            return OperationResult<Product>.success(
                message: 'Producto registrado correctamente.',
                data: product,
            );
        } catch (e) {
            return OperationResult<Product>.failure(
                message: 'Error al registrar el producto: $e',
            );
        }
    }
}

```

```

    );
  } catch (error) {
    return OperationResult<Product>.failure(
      message: 'No se pudo registrar el producto: $error',
    );
  }
}

```

```

Future<OperationResult<List<Product>>> getProducts({
  bool includeInactive = false,
}) async {
  try {
    final products = await _productRepository.findAll(
      includeInactive: includeInactive,
    );

    return OperationResult<List<Product>>.success(
      message: products.isEmpty
        ? 'No existen productos registrados.'
        : 'Productos consultados correctamente.',
      data: products,
    );
  } catch (error) {
    return OperationResult<List<Product>>.failure(
      message: 'No se pudo consultar el inventario: $error',
    );
  }
}

```

```

Future<OperationResult<Product>> getProductById(
  String id,
) async {
  if (id.trim().isEmpty) {
    return OperationResult<Product>.failure(
      message: 'El identificador del producto es obligatorio.',
    );
  }
}

```

```

final product = await _productRepository.findById(
  id.trim(),
);

```

```

if (product == null) {
  return OperationResult<Product>.failure(
    message: 'Producto no encontrado.',
  );
}

```

```

return OperationResult<Product>.success(
  message: 'Producto encontrado.',
)

```

```

        data: product,
    );
}

Future<OperationResult<List<Product>>> searchProducts({
    String query = "",
    String? category,
    ProductStatus? status,
    ProductSortOption sortOption =
        ProductSortOption.name,
    bool ascending = true,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: true,
        );

        final normalizedQuery =
            query.trim().toLowerCase();

        final filteredProducts = products.where((product) {
            final matchesQuery =
                normalizedQuery.isEmpty ||
                product.name
                    .toLowerCase()
                    .contains(normalizedQuery) ||
                product.sku
                    .toLowerCase()
                    .contains(normalizedQuery);

            final matchesCategory =
                category == null ||
                category.trim().isEmpty ||
                product.category.toLowerCase() ==
                    category.trim().toLowerCase();

            final matchesStatus =
                status == null || product.status == status;

            return matchesQuery &&
                matchesCategory &&
                matchesStatus;
        }).toList();

        _sortProducts(
            filteredProducts,
            sortOption: sortOption,
            ascending: ascending,
        );
    }
}

```

```

return OperationResult<List<Product>>.success(
    message: filteredProducts.isEmpty
        ? 'No se encontraron productos.'
        : 'Búsqueda realizada correctamente.',
    data: filteredProducts,
);
} catch (error) {
return OperationResult<List<Product>>.failure(
    message: 'No se pudo realizar la búsqueda: $error',
);
}
}
}

```

```

Future<OperationResult<Product>> updateProduct({
    required Product product,
}) async {
    final currentProduct =
        await _productRepository.findById(product.id);

    if (currentProduct == null) {
        return OperationResult<Product>.failure(
            message: 'El producto que desea editar no existe.',
        );
    }
}

```

```

final normalizedProduct = product.copyWith(
    sku: product.sku.trim().toUpperCase(),
    name: product.name.trim(),
    category: product.category.trim(),
    location: product.location.trim(),
);

```

```

final validationErrors =
    ProductValidator.validateProduct(
        normalizedProduct,
    );

```

```

if (validationErrors.isNotEmpty) {
    return OperationResult<Product>.failure(
        message: validationErrors.join('\n'),
    );
}

```

```

final duplicatedSku =
    await _productRepository.existsBySku(
        normalizedProduct.sku,
        excludingProductId: normalizedProduct.id,
    );

```

```

if (duplicatedSku) {

```

```

    return OperationResult<Product>.failure(
        message:
            'Ya existe otro producto con el SKU ${normalizedProduct.sku}.',
    );
}

try {
    await _productRepository.update(
        normalizedProduct,
    );

    return OperationResult<Product>.success(
        message: 'Producto actualizado correctamente.',
        data: normalizedProduct,
    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo actualizar el producto: $error',
    );
}
}

```

```

Future<OperationResult<Product>> deactivateProduct(
    String id,
) async {
    final product = await _productRepository.findById(id);

    if (product == null) {
        return OperationResult<Product>.failure(
            message:
                'El producto que desea dar de baja no existe.',
        );
    }

    if (!product.isActive) {
        return OperationResult<Product>.failure(
            message: 'El producto ya se encuentra dado de baja.',
        );
    }

    try {
        await _productRepository.deactivate(id);

        final inactiveProduct = product.copyWith(
            status: ProductStatus.inactive,
        );

        return OperationResult<Product>.success(
            message: 'Producto dado de baja exitosamente.',
            data: inactiveProduct,
        );
    }
}

```

```

    );
} catch (error) {
    return OperationResult<Product>.failure(
        message: 'No se pudo dar de baja el producto: $error',
    );
}
}

void _sortProducts(
    List<Product> products, {
    required ProductSortOption sortOption,
    required bool ascending,
}) {
    products.sort((first, second) {
        int comparison;

        switch (sortOption) {
            case ProductSortOption.name:
                comparison = first.name
                    .toLowerCase()
                    .compareTo(second.name.toLowerCase());
                break;

            case ProductSortOption.price:
                comparison =
                    first.price.compareTo(second.price);
                break;

            case ProductSortOption.stock:
                comparison =
                    first.stock.compareTo(second.stock);
                break;
        }

        return ascending ? comparison : -comparison;
    });
}

String _generateProductId() {
    final timestamp =
        DateTime.now().microsecondsSinceEpoch;

    return 'PRODUCT-$timestamp';
}
}

```

Pruebas Realizadas

```
group('ProductService - RQF-013: Editar Producto', () {
    late ProductService productService;

    setUp(() async {
        final repo = InMemoryProductRepository();
        productService = ProductService(productRepository: repo);
    });

    test('actualizar producto exitosamente', () async {
        final saved = await productService.registerProduct(
            sku: 'SKU-001',
            name: 'Original',
            price: 10.0,
            stock: 10,
            category: 'Cat',
            location: 'A1',
        );

        final updatedProduct = saved.data!.copyWith(
            name: 'Actualizado',
            price: 20.0,
        );

        final result = await productService.updateProduct(
            product: updatedProduct,
        );

        expect(result.isSuccess, true);
        expect(result.data!.name, 'Actualizado');
    });

    test('rechazar actualización de producto inexistente', () async {
        const nonExistent = Product(
            id: 'NO-EXISTE',
            sku: 'SKU-X',
            name: 'No existe',
            price: 10.0,
            stock: 5,
```

```

        category: 'Cat',
        location: 'A1',
    );

    final result = await productService.updateProduct(
        product: nonexistent,
    );
    expect(result.isSuccess, false);
    expect(result.message, contains('no existe'));
});

test('rechazar actualización con datos inválidos', () async {
    final saved = await productService.registerProduct(
        sku: 'SKU-001',
        name: 'Original',
        price: 10.0,
        stock: 10,
        category: 'Cat',
        location: 'A1',
    );

    final invalidProduct = saved.data!.copyWith(
        price: 0,
        name: '',
    );

    final result = await productService.updateProduct(
        product: invalidProduct,
    );
    expect(result.isSuccess, false);
    expect(result.message, contains('obligatorio'));
});

test('rechazar SKU duplicado en actualización', () async {
    await productService.registerProduct(
        sku: 'SKU-001',
        name: 'Producto 1',
        price: 10.0,
        stock: 10,
    );

```

```

        category: 'Cat',
        location: 'A1',
    );
    final saved2 = await productService.registerProduct(
        sku: 'SKU-002',
        name: 'Producto 2',
        price: 20.0,
        stock: 20,
        category: 'Cat',
        location: 'A2',
    );

    final withDuplicatedSku = saved2.data!.copyWith(sku: 'SKU-001');
    final result = await productService.updateProduct(
        product: withDuplicatedSku,
    );
    expect(result.isSuccess, false);
    expect(result.message, contains('Ya existe otro producto con el
SKU'));
  });
});

```

Resultados Obtenidos

```

~/codigoSprints/paperboost x
~/codigoSprints/paperboost x flutter test test/logic/services/product_service_test.dart --plain-name "ProductService - RQF-013: Editar Producto"
00:02 +4: All tests passed!

```

RQF-015: Configurar stock mínimo

Como se implementó

Archivo principal a testear: lib\logic\services\stock_alert_service.dart

```

import '../data/models/stock_alert.dart';
import '../data/repositories/product_repository.dart';
import '../data/repositories/stock_alert_repository.dart';

```

```

import '../results/operation_result.dart';
import '../validators/stock_alert_validator.dart';

class StockAlertService {
  StockAlertService({
    required StockAlertRepository stockAlertRepository,
    required ProductRepository productRepository,
  }) : _stockAlertRepository = stockAlertRepository,
       _productRepository = productRepository;

  final StockAlertRepository _stockAlertRepository;
  final ProductRepository _productRepository;

  Future<OperationResult<StockAlert>> createAlert({
    required String productId,
    required int minimumQuantity,
  }) async {
    final product = await _productRepository.findById(productId);
    if (product == null) {
      return OperationResult<StockAlert>.failure(
        message: 'El producto especificado no existe.',
      );
    }

    final existingAlert = await _stockAlertRepository.findByProductId(productId);
    if (existingAlert != null) {
      return OperationResult<StockAlert>.failure(
        message: 'Ya existe una alerta configurada para este producto.',
      );
    }

    final alert = StockAlert(
      id: 'ALERT- $\{DateTime.now().microsecondsSinceEpoch\}$ ',
      productId: productId,
      minimumQuantity: minimumQuantity,
      isActive: true,
    );

    final validationErrors = StockAlertValidator.validate(alert);
    if (validationErrors.isNotEmpty) {
      return OperationResult<StockAlert>.failure(
        message: validationErrors.join('\n'),
      );
    }

    try {
      await _stockAlertRepository.save(alert);
      return OperationResult<StockAlert>.success(
        message: 'Alerta de stock creada correctamente.',
        data: alert,
      );
    } catch (e) {
      return OperationResult<StockAlert>.failure(
        message: 'Error al crear la alerta de stock: $e',
      );
    }
  }
}

```

```

    );
} catch (error) {
    return OperationResult<StockAlert>.failure(
        message: 'Error al registrar la alerta: $error',
    );
}
}

Future<OperationResult<StockAlert>> updateAlert({
    required String id,
    required int minimumQuantity,
    required bool isActive,
}) async {
    final alert = await _stockAlertRepository.findById(id);
    if (alert == null) {
        return OperationResult<StockAlert>.failure(
            message: 'La alerta especificada no existe.',
        );
    }

    final updatedAlert = alert.copyWith(
        minimumQuantity: minimumQuantity,
        isActive: isActive,
    );

    final validationErrors = StockAlertValidator.validate(updatedAlert);
    if (validationErrors.isNotEmpty) {
        return OperationResult<StockAlert>.failure(
            message: validationErrors.join('\n'),
        );
    }

    try {
        await _stockAlertRepository.update(updatedAlert);
        return OperationResult<StockAlert>.success(
            message: 'Alerta de stock actualizada correctamente.',
            data: updatedAlert,
        );
    } catch (error) {
        return OperationResult<StockAlert>.failure(
            message: 'Error al actualizar la alerta: $error',
        );
    }
}

Future<OperationResult<StockAlert>> toggleAlertStatus(String id) async {
    final alert = await _stockAlertRepository.findById(id);
    if (alert == null) {
        return OperationResult<StockAlert>.failure(
            message: 'La alerta especificada no existe.',
        );
    }
}

```

```

    );
}

final updatedAlert = alert.copyWith(isActive: !alert.isActive);

try {
  await _stockAlertRepository.update(updatedAlert);
  final msg = updatedAlert.isActive
    ? 'Alerta activada correctamente.'
    : 'Alerta desactivada correctamente.';
  return OperationResult<StockAlert>.success(
    message: msg,
    data: updatedAlert,
  );
} catch (error) {
  return OperationResult<StockAlert>.failure(
    message: 'Error al cambiar estado de la alerta: $error',
  );
}
}

Future<OperationResult<List<StockAlert>>> getAlerts() async {
  try {
    final alerts = await _stockAlertRepository.findAll();
    return OperationResult<List<StockAlert>>.success(
      message: 'Alertas consultadas correctamente.',
      data: alerts,
    );
  } catch (error) {
    return OperationResult<List<StockAlert>>.failure(
      message: 'Error al consultar alertas: $error',
    );
  }
}

Future<OperationResult<StockAlert?>> getAlertByProductId(String productId) async {
  try {
    final alert = await _stockAlertRepository.findByProductId(productId);
    return OperationResult<StockAlert?>.success(
      message: 'Alerta consultada correctamente.',
      data: alert,
    );
  } catch (error) {
    return OperationResult<StockAlert?>.failure(
      message: 'Error al consultar la alerta del producto: $error',
    );
  }
}

Future<OperationResult<List<StockAlert>>> getTriggeredAlerts() async {

```

```

try {
    final alerts = await _stockAlertRepository.findAll();
    final triggered = <StockAlert>[];

    for (final alert in alerts) {
        if (alert.isActive) {
            final product = await _productRepository.findById(alert.productId);
            if (product != null && product.stock <= alert.minimumQuantity) {
                triggered.add(alert);
            }
        }
    }

    return OperationResult<List<StockAlert>>.success(
        message: 'Alertas activadas consultadas correctamente.',
        data: triggered,
    );
} catch (error) {
    return OperationResult<List<StockAlert>>.failure(
        message: 'Error al consultar alertas activadas: $error',
    );
}
}
}

```

Pruebas Realizadas

```

group('StockAlertService Tests', () {
    late StockAlertService service;
    late InMemoryStockAlertRepository alertRepository;
    late InMemoryProductRepository productRepository;
    late Product testProduct;

    setUp(() async {
        alertRepository = InMemoryStockAlertRepository();
        productRepository = InMemoryProductRepository();

        testProduct = const Product(
            id: 'PROD-001',
            sku: 'SKU-001',
            name: 'Product Test',
            price: 100.0,

```

```

        stock: 5,
        category: 'Test',
        location: 'A1',
    );

    await productRepository.save(testProduct);

    service = StockAlertService(
      stockAlertRepository: alertRepository,
      productRepository: productRepository,
    );
  });

  test('Crear alerta exitosamente', () async {
    final result = await service.createAlert(
      productId: 'PROD-001',
      minimumQuantity: 10,
    );

    expect(result.isSuccess, true);
    expect(result.data, isNotNull);
    expect(result.data!.productId, 'PROD-001');
    expect(result.data!.minimumQuantity, 10);
    expect(result.data!.isActive, true);
  });

  test('Fallar al crear alerta para producto inexistente', () async {
    final result = await service.createAlert(
      productId: 'PROD-NONEXISTENT',
      minimumQuantity: 10,
    );

    expect(result.isSuccess, false);
    expect(result.message, contains('no existe'));
  });

  test('Fallar al crear alerta duplicada para el mismo producto', ()
  async {
    await service.createAlert(

```

```
        productId: 'PROD-001',
        minimumQuantity: 10,
    );

    final result = await service.createAlert(
        productId: 'PROD-001',
        minimumQuantity: 5,
    );

    expect(result.isSuccess, false);
    expect(result.message, contains('Ya existe una alerta'));
});

test('Actualizar alerta exitosamente', () async {
    final registerResult = await service.createAlert(
        productId: 'PROD-001',
        minimumQuantity: 10,
    );

    final alertId = registerResult.data!.id;

    final updateResult = await service.updateAlert(
        id: alertId,
        minimumQuantity: 15,
        isActive: false,
    );

    expect(updateResult.isSuccess, true);
    expect(updateResult.data!.minimumQuantity, 15);
    expect(updateResult.data!.isActive, false);
});

test('Fallar al crear alerta con cantidad mínima inválida', () async {
    final result = await service.createAlert(
        productId: 'PROD-001',
        minimumQuantity: -1,
    );
}
```

```

        expect(result.isSuccess, false);
        expect(result.message, contains('cantidad mínima no puede ser
negativa'));
    });

    test('Fallar al actualizar alerta con datos inválidos', () async {
        final registerResult = await service.createAlert(
            productId: 'PROD-001',
            minimumQuantity: 10,
        );

        final updateResult = await service.updateAlert(
            id: registerResult.data!.id,
            minimumQuantity: -1,
            isActive: true,
        );
        expect(updateResult.isSuccess, false);
        expect(updateResult.message, contains('cantidad mínima no puede ser
negativa'));
    });
});

```

Resultados Obtenidos



RQF-016: Administrar alertas de stock

Como se implementó

Archivo principal a testear: lib\logic\controllers\stock_alert_controller.dart

```

import '../data/models/stock_alert.dart';
import '../results/operation_result.dart';
import '../services/stock_alert_service.dart';

class StockAlertController {

```

```

StockAlertController({
    required StockAlertService stockAlertService,
}) : _stockAlertService = stockAlertService;

final StockAlertService _stockAlertService;

Future<OperationResult<StockAlert>> createAlert({
    required String productId,
    required int minimumQuantity,
}) {
    return _stockAlertService.createAlert(
        productId: productId,
        minimumQuantity: minimumQuantity,
    );
}

Future<OperationResult<StockAlert>> updateAlert({
    required String id,
    required int minimumQuantity,
    required bool isActive,
}) {
    return _stockAlertService.updateAlert(
        id: id,
        minimumQuantity: minimumQuantity,
        isActive: isActive,
    );
}

Future<OperationResult<StockAlert>> toggleAlertStatus(String id) {
    return _stockAlertService.toggleAlertStatus(id);
}

Future<OperationResult<List<StockAlert>>> getAlerts() {
    return _stockAlertService.getAlerts();
}

Future<OperationResult<StockAlert?>> getAlertByProductId(String productId) {
    return _stockAlertService.getAlertByProductId(productId);
}

Future<OperationResult<List<StockAlert>>> getTriggeredAlerts() {
    return _stockAlertService.getTriggeredAlerts();
}
}

```

Pruebas Realizadas

```
test('Cambiar estado (toggle) de alerta', () async {
  final registerResult = await service.createAlert(
    productId: 'PROD-001',
    minimumQuantity: 10,
  );

  final alertId = registerResult.data!.id;

  final toggle1 = await service.toggleAlertStatus(alertId);
  expect(toggle1.isSuccess, true);
  expect(toggle1.data!.isActive, false);

  final toggle2 = await service.toggleAlertStatus(alertId);
  expect(toggle2.isSuccess, true);
  expect(toggle2.data!.isActive, true);
});

test('Obtener alertas disparadas (triggered)', () async {
  // testProduct tiene stock = 5.
  // Creamos alerta activa con mínimo 10 (debería dispararse)
  await service.createAlert(
    productId: 'PROD-001',
    minimumQuantity: 10,
  );

  final triggeredResult = await service.getTriggeredAlerts();

  expect(triggeredResult.isSuccess, true);
  expect(triggeredResult.data!.length, 1);
  expect(triggeredResult.data!.first.productId, 'PROD-001');

  // Creamos otro producto con stock 20 y alerta 5 (no debería
  dispararse)
  const p2 = Product(
    id: 'PROD-002',
    sku: 'SKU-002',
    name: 'Product 2',
    price: 50.0,
    stock: 20,
```

```

        category: 'Test',
        location: 'A2',
    );
    await productRepository.save(p2);
    await service.createAlert(
        productId: 'PROD-002',
        minimumQuantity: 5,
    );

    final triggeredResult2 = await service.getTriggeredAlerts();
    expect(triggeredResult2.data!.length, 1); // Sigue siendo sólo el
    primero
  });

```

Resultados Obtenidos



RQF-019: Consultar historial de ventas

Como se implementó

Archivo principal a testear: lib\logic\services\sale_service.dart

```

import '../data/models/sale.dart';
import '../data/models/sale_item.dart';
import '../data/repositories/product_repository.dart';
import '../data/repositories/sale_repository.dart';
import '../observers/stock_change_notifier.dart';
import '../results/operation_result.dart';
import '../validators/sale_validator.dart';

class SaleService {
  SaleService({
    required SaleRepository saleRepository,
    required ProductRepository productRepository,
    StockChangeNotifier? stockChangeNotifier,
  }) : _saleRepository = saleRepository,
       _productRepository = productRepository,
       _stockChangeNotifier =
         stockChangeNotifier ?? StockChangeNotifier.instance;

```

```
static const double _taxRate = 0.15; // 15% IVA (ajustar según país)
static int _saleIdCounter = 0;
```

```
final SaleRepository _saleRepository;
final ProductRepository _productRepository;
final StockChangeNotifier _stockChangeNotifier;
```

```
Future<OperationResult<Sale>> createSale({
  required List<SaleItem> items,
  required PaymentMethod paymentMethod,
  String observations = "",
  String customerName = "",
  String customerEmail = "",
}) async {
  // Validar items
  final itemErrors = SaleValidator.validateSaleItems(items);
  if (itemErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: itemErrors.join('\n'),
    );
  }

  // Validar disponibilidad de stock
  final stockErrors = SaleValidator.validateStockForAllItems(items);
  if (stockErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: stockErrors.join('\n'),
    );
  }

  // Validar información del cliente (opcional)
  final customerErrors = SaleValidator.validateCustomerInfo(
    customerName.isEmpty ? null : customerName,
    customerEmail.isEmpty ? null : customerEmail,
  );
  if (customerErrors.isNotEmpty) {
    return OperationResult<Sale>.failure(
      message: customerErrors.join('\n'),
    );
  }

  try {
    // Calcular totales
    final subtotal = items.fold<double>(
      0,
      (sum, item) => sum + item.subtotal,
    );

    final taxAmount = subtotal * _taxRate;
```

```

final total = subtotal + taxAmount;

// Generar número de venta
final nextNumber = await _saleRepository.getNextSaleNumber();
final saleNumber = _formatSaleNumber(nextNumber);

// Crear venta
final sale = Sale(
  id: _generateSaleId(),
  saleNumber: saleNumber,
  items: items,
  subtotal: subtotal,
  taxAmount: taxAmount,
  total: total,
  paymentMethod: paymentMethod,
  status: SaleStatus.completed,
  createdAt: DateTime.now(),
  observations: observations.trim(),
  customerName: customerName.trim(),
  customerEmail: customerEmail.trim(),
);

// Guardar venta
await _saleRepository.save(sale);

// Descontar stock y notificar
await _decrementProductsStock(items);

return OperationResult<Sale>.success(
  message: 'Nota de venta creada correctamente.',
  data: sale,
);
} catch (error) {
  return OperationResult<Sale>.failure(
    message: 'No se pudo crear la nota de venta: $error',
  );
}
}

Future<OperationResult<Sale>> completeSale(String saleId) async {
  try {
    final sale = await _saleRepository.findById(saleId);

    if (sale == null) {
      return OperationResult<Sale>.failure(
        message: 'No se encontró la venta.',
      );
    }

    if (sale.isCompleted) {

```

```

        return OperationResult<Sale>.success(
            message: 'La venta ya estaba completada.',
            data: sale,
        );
    }

    if (!sale.isPending) {
        return OperationResult<Sale>.failure(
            message: 'Solo se pueden completar ventas pendientes.',
        );
    }

    final completedSale =
        sale.copyWith(status: SaleStatus.completed);

    await _saleRepository.update(completedSale);

    return OperationResult<Sale>.success(
        message: 'Venta completada correctamente.',
        data: completedSale,
    );
} catch (error) {
    return OperationResult<Sale>.failure(
        message: 'No se pudo completar la venta: $error',
    );
}
}

Future<OperationResult<void>> cancelSale(String saleId) async {
    try {
        final sale = await _saleRepository.findById(saleId);

        if (sale == null) {
            return OperationResult<void>.failure(
                message: 'No se encontró la venta.',
            );
        }

        if (sale.isCancelled) {
            return OperationResult<void>.failure(
                message: 'La venta ya está cancelada.',
            );
        }

        // Revertir stock
        await _restoreProductsStock(sale.items);

        // Cancelar venta
        await _saleRepository.cancel(saleId);
    }
}

```

```

        return OperationResult<void>.success(
            message: 'Venta cancelada correctamente.',
        );
    } catch (error) {
        return OperationResult<void>.failure(
            message: 'No se pudo cancelar la venta: $error',
        );
    }
}

```

```

Future<OperationResult<List<Sale>>> getSales({
    bool includeCancelled = false,
}) async {
    try {
        final sales = await _saleRepository.findAll(
            includeCompleted: true,
            includeCancelled: includeCancelled,
        );

        return OperationResult<List<Sale>>.success(
            message: sales.isEmpty
                ? 'No hay ventas registradas.'
                : 'Ventas consultadas correctamente.',
            data: sales,
        );
    } catch (error) {
        return OperationResult<List<Sale>>.failure(
            message: 'No se pudo consultar las ventas: $error',
        );
    }
}

```

```

Future<OperationResult<Sale>> getSaleById(String saleId) async {
    try {
        final sale = await _saleRepository.findById(saleId);

        if (sale == null) {
            return OperationResult<Sale>.failure(
                message: 'No se encontró la venta.',
            );
        }

        return OperationResult<Sale>.success(
            message: 'Venta consultada correctamente.',
            data: sale,
        );
    } catch (error) {
        return OperationResult<Sale>.failure(
            message: 'No se pudo consultar la venta: $error',
        );
    }
}

```

```
}  
}
```

```
Future<OperationResult<List<Sale>>> searchSales({  
    String query = "",  
    SaleStatus? status,  
}) async {  
    try {  
        final sales = await _saleRepository.findAll(  
            includeCompleted: true,  
            includeCancelled: true,  
        );  
  
        final normalizedQuery = query.trim().toLowerCase();  
  
        final filteredSales = sales.where((sale) {  
            final matchesQuery = normalizedQuery.isEmpty ||  
                sale.saleNumber.toLowerCase().contains(normalizedQuery) ||  
                sale.customerName.toLowerCase().contains(normalizedQuery) ||  
                sale.customerEmail.toLowerCase().contains(normalizedQuery);  
  
            final matchesStatus = status == null || sale.status == status;  
  
            return matchesQuery && matchesStatus;  
        }).toList();  
  
        return OperationResult<List<Sale>>.success(  
            message: filteredSales.isEmpty  
                ? 'No se encontraron ventas.'  
                : 'Ventas consultadas correctamente.',  
            data: filteredSales,  
        );  
    } catch (error) {  
        return OperationResult<List<Sale>>.failure(  
            message: 'No se pudo realizar la búsqueda: $error',  
        );  
    }  
}
```

```
Future<void> _decrementProductsStock(List<SaleItem> items) async {  
    for (final item in items) {  
        final product = await _productRepository.findById(  
            item.product.id,  
        );  
  
        if (product != null) {  
            final oldStock = product.stock;  
            final newStock = product.stock - item.quantity;  
            final updatedProduct = product.copyWith(stock: newStock);  
        }  
    }  
}
```

```

        await _productRepository.update(updatedProduct);

        // Notificar cambio de stock
        _stockChangeNotifier.notifyStockChanged(
            product,
            oldStock,
            newStock,
        );

        // Si el stock llega a 0, notificar que no hay disponibilidad
        if (newStock == 0) {
            _stockChangeNotifier.notifyStockUnavailable(updatedProduct);
        }
    }
}

Future<void> _restoreProductsStock(List<SaleItem> items) async {
    for (final item in items) {
        final product = await _productRepository.findById(
            item.product.id,
        );

        if (product != null) {
            final oldStock = product.stock;
            final newStock = product.stock + item.quantity;
            final updatedProduct = product.copyWith(stock: newStock);

            await _productRepository.update(updatedProduct);

            _stockChangeNotifier.notifyStockChanged(
                product,
                oldStock,
                newStock,
            );
        }
    }
}

String _formatSaleNumber(int number) {
    return 'VTA-${number.toString().padLeft(6, '0')}';
}

String _generateSaleId() {
    _saleIdCounter++;
    return 'SALE-${_saleIdCounter.toString().padLeft(4, '0')}';
}
}

```

Pruebas Realizadas

```
group('SaleService - Consultar Ventas', () {
  late SaleService saleService;
  late InMemorySaleRepository saleRepository;
  late InMemoryProductRepository productRepository;

  setUp(() async {
    saleRepository = InMemorySaleRepository();
    productRepository = InMemoryProductRepository();

    saleService = SaleService(
      saleRepository: saleRepository,
      productRepository: productRepository,
    );
  });

  test('getSaleById retorna failure para venta inexistente', () async {
    final result = await saleService.getSaleById('NO-EXISTE');
    expect(result.isSuccess, false);
    expect(result.message, contains('No se encontró la venta'));
  });

  test('getSaleById retorna venta exitosamente', () async {
    const product = Product(
      id: 'PROD-001',
      sku: 'SKU-001',
      name: 'Product Test',
      price: 100.0,
      stock: 50,
      category: 'Test',
      location: 'A1',
    );

    await productRepository.save(product);

    final item = SaleItem(
      id: 'ITEM-001',
      product: product,
```

```

        quantity: 1,
        unitPrice: 100.0,
    );

    final createResult = await saleService.createSale(
      items: [item],
      paymentMethod: PaymentMethod.cash,
    );

    final result = await
saleService.getSaleById(createResult.data!.id);
    expect(result.isSuccess, true);
    expect(result.data!.id, createResult.data!.id);
  });

  test('getSales retorna mensaje si no hay ventas', () async {
    final result = await saleService.getSales();
    expect(result.isSuccess, true);
    expect(result.message, contains('No hay ventas registradas'));
  });
});

```

Resultados Obtenidos

```

.../codigoSprints/paperboost ) flutter test test/logic/services/sale_service_test.dart --plain-name "SaleService - Consultar Ventas"
00:03 +3: All tests passed!

```

RQF-020: Buscar y consultar detalle de venta

Como se implementó

Archivo principal a testear: lib\logic\services\sale_service.dart

```

import '../data/models/sale.dart';
import '../data/models/sale_item.dart';
import '../data/repositories/product_repository.dart';
import '../data/repositories/sale_repository.dart';

```

```

import '../observers/stock_change_notifier.dart';
import '../results/operation_result.dart';
import '../validators/sale_validator.dart';

class SaleService {
  SaleService({
    required SaleRepository saleRepository,
    required ProductRepository productRepository,
    StockChangeNotifier? stockChangeNotifier,
  }) : _saleRepository = saleRepository,
       _productRepository = productRepository,
       _stockChangeNotifier =
         stockChangeNotifier ?? StockChangeNotifier.instance;

  static const double _taxRate = 0.15; // 15% IVA (ajustar según país)
  static int _saleIdCounter = 0;

  final SaleRepository _saleRepository;
  final ProductRepository _productRepository;
  final StockChangeNotifier _stockChangeNotifier;

  Future<OperationResult<Sale>> createSale({
    required List<SaleItem> items,
    required PaymentMethod paymentMethod,
    String observations = "",
    String customerName = "",
    String customerEmail = "",
  }) async {
    // Validar items
    final itemErrors = SaleValidator.validateSaleItems(items);
    if (itemErrors.isNotEmpty) {
      return OperationResult<Sale>.failure(
        message: itemErrors.join("\n"),
      );
    }

    // Validar disponibilidad de stock
    final stockErrors = SaleValidator.validateStockForAllItems(items);
    if (stockErrors.isNotEmpty) {
      return OperationResult<Sale>.failure(
        message: stockErrors.join("\n"),
      );
    }

    // Validar información del cliente (opcional)
    final customerErrors = SaleValidator.validateCustomerInfo(
      customerName.isEmpty ? null : customerName,
      customerEmail.isEmpty ? null : customerEmail,
    );
    if (customerErrors.isNotEmpty) {

```

```

    return OperationResult<Sale>.failure(
        message: customerErrors.join("\n"),
    );
}

try {
    // Calcular totales
    final subtotal = items.fold<double>(
        0,
        (sum, item) => sum + item.subtotal,
    );

    final taxAmount = subtotal * _taxRate;
    final total = subtotal + taxAmount;

    // Generar número de venta
    final nextNumber = await _saleRepository.getNextSaleNumber();
    final saleNumber = _formatSaleNumber(nextNumber);

    // Crear venta
    final sale = Sale(
        id: _generateSaleId(),
        saleNumber: saleNumber,
        items: items,
        subtotal: subtotal,
        taxAmount: taxAmount,
        total: total,
        paymentMethod: paymentMethod,
        status: SaleStatus.completed,
        createdAt: DateTime.now(),
        observations: observations.trim(),
        customerName: customerName.trim(),
        customerEmail: customerEmail.trim(),
    );

    // Guardar venta
    await _saleRepository.save(sale);

    // Descontar stock y notificar
    await _decrementProductsStock(items);

    return OperationResult<Sale>.success(
        message: 'Nota de venta creada correctamente.',
        data: sale,
    );
} catch (error) {
    return OperationResult<Sale>.failure(
        message: 'No se pudo crear la nota de venta: $error',
    );
}

```

```
}
```

```
Future<OperationResult<Sale>> completeSale(String saleId) async {
```

```
  try {
```

```
    final sale = await _saleRepository.findById(saleId);
```

```
    if (sale == null) {
```

```
      return OperationResult<Sale>.failure(
```

```
        message: 'No se encontró la venta.',
```

```
      );
```

```
    }
```

```
    if (sale.isCompleted) {
```

```
      return OperationResult<Sale>.success(
```

```
        message: 'La venta ya estaba completada.',
```

```
        data: sale,
```

```
      );
```

```
    }
```

```
    if (!sale.isPending) {
```

```
      return OperationResult<Sale>.failure(
```

```
        message: 'Solo se pueden completar ventas pendientes.',
```

```
      );
```

```
    }
```

```
    final completedSale =
```

```
      sale.copyWith(status: SaleStatus.completed);
```

```
    await _saleRepository.update(completedSale);
```

```
    return OperationResult<Sale>.success(
```

```
      message: 'Venta completada correctamente.',
```

```
      data: completedSale,
```

```
    );
```

```
  } catch (error) {
```

```
    return OperationResult<Sale>.failure(
```

```
      message: 'No se pudo completar la venta: $error',
```

```
    );
```

```
  }
```

```
}
```

```
Future<OperationResult<void>> cancelSale(String saleId) async {
```

```
  try {
```

```
    final sale = await _saleRepository.findById(saleId);
```

```
    if (sale == null) {
```

```
      return OperationResult<void>.failure(
```

```
        message: 'No se encontró la venta.',
```

```
      );
```

```
    }
```

```

    if (sale.isCancelled) {
        return OperationResult<void>.failure(
            message: 'La venta ya está cancelada.',
        );
    }

    // Revertir stock
    await _restoreProductsStock(sale.items);

    // Cancelar venta
    await _saleRepository.cancel(saleId);

    return OperationResult<void>.success(
        message: 'Venta cancelada correctamente.',
    );
} catch (error) {
    return OperationResult<void>.failure(
        message: 'No se pudo cancelar la venta: $error',
    );
}
}

Future<OperationResult<List<Sale>>> getSales({
    bool includeCancelled = false,
}) async {
    try {
        final sales = await _saleRepository.findAll(
            includeCompleted: true,
            includeCancelled: includeCancelled,
        );

        return OperationResult<List<Sale>>.success(
            message: sales.isEmpty
                ? 'No hay ventas registradas.'
                : 'Ventas consultadas correctamente.',
            data: sales,
        );
    } catch (error) {
        return OperationResult<List<Sale>>.failure(
            message: 'No se pudo consultar las ventas: $error',
        );
    }
}

Future<OperationResult<Sale>> getSaleById(String saleId) async {
    try {
        final sale = await _saleRepository.findById(saleId);

        if (sale == null) {

```

```

        return OperationResult<Sale>.failure(
            message: 'No se encontró la venta.',
        );
    }

    return OperationResult<Sale>.success(
        message: 'Venta consultada correctamente.',
        data: sale,
    );
} catch (error) {
    return OperationResult<Sale>.failure(
        message: 'No se pudo consultar la venta: $error',
    );
}
}

Future<OperationResult<List<Sale>>> searchSales({
    String query = "",
    SaleStatus? status,
}) async {
    try {
        final sales = await _saleRepository.findAll(
            includeCompleted: true,
            includeCancelled: true,
        );

        final normalizedQuery = query.trim().toLowerCase();

        final filteredSales = sales.where((sale) {
            final matchesQuery = normalizedQuery.isEmpty ||
                sale.saleNumber.toLowerCase().contains(normalizedQuery) ||
                sale.customerName.toLowerCase().contains(normalizedQuery) ||
                sale.customerEmail.toLowerCase().contains(normalizedQuery);

            final matchesStatus = status == null || sale.status == status;

            return matchesQuery && matchesStatus;
        }).toList();

        return OperationResult<List<Sale>>.success(
            message: filteredSales.isEmpty
                ? 'No se encontraron ventas.'
                : 'Ventas consultadas correctamente.',
            data: filteredSales,
        );
    } catch (error) {
        return OperationResult<List<Sale>>.failure(
            message: 'No se pudo realizar la búsqueda: $error',
        );
    }
}

```

```
}
```

```
Future<void> _decrementProductsStock(List<SaleItem> items) async {  
  for (final item in items) {  
    final product = await _productRepository.findById(  
      item.product.id,  
    );  
  
    if (product != null) {  
      final oldStock = product.stock;  
      final newStock = product.stock - item.quantity;  
      final updatedProduct = product.copyWith(stock: newStock);  
  
      await _productRepository.update(updatedProduct);  
  
      // Notificar cambio de stock  
      _stockChangeNotifier.notifyStockChanged(  
        product,  
        oldStock,  
        newStock,  
      );  
  
      // Si el stock llega a 0, notificar que no hay disponibilidad  
      if (newStock == 0) {  
        _stockChangeNotifier.notifyStockUnavailable(updatedProduct);  
      }  
    }  
  }  
}
```

```
Future<void> _restoreProductsStock(List<SaleItem> items) async {  
  for (final item in items) {  
    final product = await _productRepository.findById(  
      item.product.id,  
    );  
  
    if (product != null) {  
      final oldStock = product.stock;  
      final newStock = product.stock + item.quantity;  
      final updatedProduct = product.copyWith(stock: newStock);  
  
      await _productRepository.update(updatedProduct);  
  
      _stockChangeNotifier.notifyStockChanged(  
        product,  
        oldStock,  
        newStock,  
      );  
    }  
  }  
}
```

```

}

String _formatSaleNumber(int number) {
  return 'VTA-${number.toString().padLeft(6, '0')};
}

String _generateSaleId() {
  _saleIdCounter++;
  return 'SALE-${_saleIdCounter.toString().padLeft(4, '0')};
}
}

```

Pruebas Realizadas

```

group('SaleService - Búsqueda de Ventas', () {
  late SaleService saleService;
  late InMemorySaleRepository saleRepository;
  late InMemoryProductRepository productRepository;
  late Product testProduct;

  setUp(() async {
    saleRepository = InMemorySaleRepository();
    productRepository = InMemoryProductRepository();

    testProduct = const Product(
      id: 'PROD-001',
      sku: 'SKU-001',
      name: 'Product Test',
      price: 100.0,
      stock: 50,
      category: 'Test',
      location: 'A1',
    );

    await productRepository.save(testProduct);

    saleService = SaleService(
      saleRepository: saleRepository,
      productRepository: productRepository,

```

```
);

// Crear algunas ventas
final item = SaleItem(
  id: 'ITEM-001',
  product: testProduct,
  quantity: 1,
  unitPrice: 100.0,
);

await saleService.createSale(
  items: [item],
  paymentMethod: PaymentMethod.cash,
  customerName: 'Juan Perez',
  customerEmail: 'juan@example.com',
  observations: 'First sale',
);

await saleService.createSale(
  items: [item],
  paymentMethod: PaymentMethod.card,
  customerName: 'Maria Gomez',
  customerEmail: 'maria@example.com',
  observations: 'Second sale',
);
});

test('Buscar venta por número de nota exacto/parcial', () async {
  // Debería encontrar VTA-000001
  final result = await saleService.searchSales(query: 'VTA-000001');

  expect(result.isSuccess, true);
  expect(result.data!.length, 1);
  expect(result.data!.first.customerName, 'Juan Perez');
});

test('Buscar venta por nombre de cliente', () async {
  final result = await saleService.searchSales(query: 'maria');
```

```
    expect(result.isSuccess, true);
    expect(result.data!.length, 1);
    expect(result.data!.first.customerName, 'Maria Gomez');
  });

  test('Buscar venta por email de cliente', () async {
    final result = await saleService.searchSales(query:
'juan@example.com');

    expect(result.isSuccess, true);
    expect(result.data!.length, 1);
    expect(result.data!.first.customerName, 'Juan Perez');
  });

  test('Buscar venta con consulta vacía retorna todas', () async {
    final result = await saleService.searchSales(query: '');

    expect(result.isSuccess, true);
    expect(result.data!.length, 2);
  });

  test('Filtrar por estado de venta', () async {
    final allSalesResult = await saleService.getSales(includeCancelled:
true);
    final saleId = allSalesResult.data!.first.id;

    // Cancelamos una venta
    await saleService.cancelSale(saleId);

    final completedSearch = await saleService.searchSales(
      status: SaleStatus.completed,
    );
    final cancelledSearch = await saleService.searchSales(
      status: SaleStatus.cancelled,
    );

    expect(completedSearch.data!.length, 1);
    expect(cancelledSearch.data!.length, 1);
  });
```

```
});
```

Resultados Obtenidos

```
.../codigoSprints/paperboost ) flutter test test/logic/services/sale_search_test.dart --plain-name "SaleService - Búsqueda de Ventas"
00:02 +5: All tests passed!
```

RQF-021: Visualizar inventario crítico

Como se implementó

Archivo principal a testear: lib\presentation\pages\inventory_page.dart

```
import 'package:flutter/material.dart';

import '../data/models/product.dart';
import '../data/models/stock_alert.dart';
import '../logic/controllers/auth_controller.dart';
import '../logic/controllers/product_controller.dart';
import '../logic/controllers/stock_alert_controller.dart';
import '../logic/observers/stock_observer.dart';
import '../logic/observers/stock_change_notifier.dart';
import '../logic/results/operation_result.dart';
import '../logic/services/product_service.dart';
import '../widgets/product_card.dart';
import 'product_form_page.dart';

class InventoryPage extends StatefulWidget {
  const InventoryPage({
    required this.authController,
    required this.productController,
    required this.stockAlertController,
    this.onLogout,
    this.onNavigateToSales,
    super.key,
  });

  final AuthController authController;
  final ProductController productController;
  final StockAlertController stockAlertController;
  final VoidCallback? onLogout;
  final VoidCallback? onNavigateToSales;

  @override
```

```

    State<InventoryPage> createState() => _InventoryPageState();
}

class _InventoryPageState extends State<InventoryPage>
    with SingleTickerProviderStateMixin
    implements StockObserver {
    final _searchController = TextEditingController();
    late TabController _tabController;

    List<Product> _products = [];
    List<String> _categories = [];
    Map<String, StockAlert> _alerts = {};

    String _selectedCategory = "";
    String _selectedStatus = 'active';

    ProductSortOption _sortOption = ProductSortOption.name;

    bool _ascending = true;
    bool _isLoading = true;

    @override
    void initState() {
        super.initState();
        _tabController = TabController(length: 2, vsync: this);
        StockChangeNotifier.instance.attach(this);
        _reloadInventory();
    }

    @override
    void dispose() {
        _tabController.dispose();
        StockChangeNotifier.instance.detach(this);
        _searchController.dispose();
        super.dispose();
    }

    @override
    void onStockChanged(Product product, int oldStock, int newStock) {
        if (mounted) {
            _reloadInventory();
        }
    }

    @override
    void onStockUnavailable(Product product) {
        if (mounted) {
            _reloadInventory();
        }
    }
}

```

```

ProductStatus? get _statusFilter {
  switch (_selectedStatus) {
    case 'active':
      return ProductStatus.active;
    case 'inactive':
      return ProductStatus.inactive;
    default:
      return null;
  }
}

Future<void> _reloadInventory() async {
  setState(() {
    _isLoading = true;
  });

  final allResult = await widget.productController.getProducts(
    includeInactive: true,
  );

  final categories = <String>{};

  for (final product in allResult.data ?? <Product>[]) {
    categories.add(product.category);
  }

  final searchResult = await widget.productController.searchProducts(
    query: _searchController.text,
    category: _selectedCategory.isEmpty ? null : _selectedCategory,
    status: _statusFilter,
    sortOption: _sortOption,
    ascending: _ascending,
  );

  final alertsResult = await widget.stockAlertController.getAlerts();
  final Map<String, StockAlert> loadedAlerts = {};
  for (final alert in alertsResult.data ?? <StockAlert>[]) {
    loadedAlerts[alert.productId] = alert;
  }

  if (!mounted) {
    return;
  }

  setState(() {
    _categories = categories.toList()..sort();
    _products = searchResult.data ?? <Product>[];
    _alerts = loadedAlerts;
    _isLoading = false;
  });
}

```

```

    });
  }

Future<void> _openProductForm({
  Product? product,
}) async {
  final changed = await Navigator.push<bool>(
    context,
    MaterialPageRoute(
      builder: (_) => ProductFormPage(
        productController: widget.productController,
        product: product,
      ),
    ),
  );

  if (changed == true) {
    await _reloadInventory();
  }
}

Future<void> _confirmDeactivate(
  Product product,
) async {
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (dialogContext) {
      return AlertDialog(
        title: const Text('Confirmar baja'),
        content: Text(
          '¿Desea dar de baja el producto '
          '"${product.name}"?',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(
                dialogContext,
                false,
              );
            },
            child: const Text('Cancelar'),
          ),
          FilledButton(
            onPressed: () {
              Navigator.pop(
                dialogContext,
                true,
              );
            },
          ),
        ],
      );
    },
  );
}

```

```

        child: const Text('Confirmar'),
      ),
    ],
  );
},
);

if (confirmed != true) {
  return;
}

final result = await widget.productController.deactivateProduct(product.id);

if (!mounted) {
  return;
}

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text(result.message),
    backgroundColor:
      result.isSuccess ? null : Theme.of(context).colorScheme.error,
  ),
);

if (result.isSuccess) {
  await _reloadInventory();
}
}

void _openConfigureAlert(Product product) async {
  final alert = _alerts[product.id];
  final isNew = alert == null;

  final quantityController = TextEditingController(
    text: alert?.minimumQuantity.toString() ?? '5',
  );
  bool isActive = alert?.isActive ?? true;

  final result = await showDialog<bool>(
    context: context,
    builder: (dialogContext) {
      return StatefulBuilder(
        builder: (context, setStateDialog) {
          return AlertDialog(
            title: Text('Configurar Alerta - ${product.name}'),
            content: SingleChildScrollView(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [

```

```

        const Text(
          'Define la cantidad mínima para notificar cuando las existencias alcancen o
bajen de este límite.',
          style: TextStyle(fontSize: 13),
        ),
        const SizedBox(height: 16),
        TextField(
          controller: quantityController,
          keyboardType: TextInputType.number,
          decoration: const InputDecoration(
            labelText: 'Cantidad Mínima',
            border: OutlineInputBorder(),
          ),
        ),
        const SizedBox(height: 12),
        SwitchListTile(
          contentPadding: EdgeInsets.zero,
          title: const Text('Alerta Activa'),
          subtitle:
            const Text('Habilita/deshabilita notificaciones'),
          value: isActive,
          onChanged: (val) {
            setStateDialog(() {
              isActive = val;
            });
          },
        ),
      ],
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(dialogContext, false),
        child: const Text('Cancelar'),
      ),
      FilledButton(
        onPressed: () => Navigator.pop(dialogContext, true),
        child: const Text('Guardar'),
      ),
    ],
  );
};

if (result != true) return;

final minQty = int.tryParse(quantityController.text) ?? -1;
if (minQty < 0) {

```

```

    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
                content:
                    Text('Cantidad mínima inválida. Debe ser mayor o igual a 0.'),
                backgroundColor: Colors.red,
            ),
        );
    }
    return;
}

OperationResult<StockAlert> opResult;
if (isNew) {
    opResult = await widget.stockAlertController.createAlert(
        productId: product.id,
        minimumQuantity: minQty,
    );
} else {
    opResult = await widget.stockAlertController.updateAlert(
        id: alert.id,
        minimumQuantity: minQty,
        isActive: isActive,
    );
}

if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
            content: Text(opResult.message),
            backgroundColor: opResult.isSuccess ? Colors.green : Colors.red,
        ),
    );
    if (opResult.isSuccess) {
        _reloadInventory();
    }
}

void _showTriggeredAlertsDialog() {
    final criticalProducts = _products.where((product) {
        final alert = _alerts[product.id];
        return product.isActive &&
            alert != null &&
            alert.isActive &&
            product.stock <= alert.minimumQuantity;
    }).toList();

    showDialog(
        context: context,

```

```

builder: (context) {
  return AlertDialog(
    title: Row(
      children: const [
        Icon(Icons.warning, color: Colors.amber),
        SizedBox(width: 8),
        Expanded(
          child: Text(
            'Alertas de Stock Crítico',
            overflow: TextOverflow.ellipsis,
          ),
        ),
      ],
    ),
    content: criticalProducts.isEmpty
      ? const Text(
          'No hay productos con existencias críticas actualmente.')
      : SizedBox(
          width: double.maxFinite,
          child: ListView.builder(
            shrinkWrap: true,
            itemCount: criticalProducts.length,
            itemBuilder: (context, index) {
              final product = criticalProducts[index];
              final alert = _alerts[product.id]!;
              return Padding(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Text(
                  '⚠️ ${product.name} - SKU: ${product.sku} (Stock: ${product.stock}, Mín:
                    ${alert.minimumQuantity})',
                  style: const TextStyle(fontSize: 12),
                  overflow: TextOverflow.ellipsis,
                ),
              );
            },
          ),
        ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cerrar'),
      ),
    ],
  );
}

```

```

Widget _buildNotificationBell() {
  final criticalCount = _products.where((product) {

```

```

        final alert = _alerts[product.id];
        return product.isActive &&
            alert != null &&
            alert.isActive &&
            product.stock <= alert.minimumQuantity;
    }).length;

    if (criticalCount == 0) {
        return const IconButton(
            icon: Icon(Icons.notifications_none),
            onPressed: null,
        );
    }

    return Badge(
        label: Text(criticalCount.toString()),
        backgroundColor: Colors.red,
        child: IconButton(
            icon: const Icon(Icons.notifications_active, color: Colors.amber),
            onPressed: _showTriggeredAlertsDialog,
        ),
    );
}

void _logout() {
    widget.onLogout?.call();
}

@override
Widget build(BuildContext context) {
    final isMobile = MediaQuery.of(context).size.width < 600;

    return Scaffold(
        appBar: AppBar(
            title: const Text('Inventario PaperBoost'),
            bottom: TabBar(
                controller: _tabController,
                tabs: const [
                    Tab(icon: Icon(Icons.inventory), text: 'Todos'),
                    Tab(icon: Icon(Icons.warning_amber), text: 'Crítico'),
                ],
            ),
        actions: [
            _buildNotificationBell(),
            if (isMobile)
                PopupMenuButton<String>(
                    icon: const Icon(Icons.more_vert),
                    onPressed: (value) {
                        if (value == 'logout') _logout();
                    },
                ),
        ],
    );
}

```

```

itemBuilder: (_) => [
  PopupMenuItem(
    value: 'logout',
    child: Row(
      children: const [
        Icon(Icons.logout, size: 20),
        SizedBox(width: 8),
        Expanded(child: Text('Cerrar sesión')),
      ],
    ),
  ),
],
)
else ...[
  IconButton(
    tooltip: 'Nueva venta',
    onPressed: widget.onNavigateToSales,
    icon: const Icon(Icons.receipt_long),
  ),
  IconButton(
    tooltip: 'Cerrar sesión',
    onPressed: _logout,
    icon: const Icon(Icons.logout),
  ),
],
),
floatingActionButton: isMobile
? FloatingActionButton(
  onPressed: () => _openProductForm(),
  child: const Icon(Icons.add),
)
: FloatingActionButton.extended(
  onPressed: () => _openProductForm(),
  icon: const Icon(Icons.add),
  label: const Text('Registrar producto'),
),
body: TabBarView(
  controller: _tabController,
  children: [
    Padding(
      padding: EdgeInsets.all(isMobile ? 12 : 20),
      child: Column(
        children: [
          if (!isMobile) ...[
            _buildQuickActions(),
            const SizedBox(height: 16),
          ],
          _buildFilters(),
          const SizedBox(height: 12),

```

```

        Expanded(
          child: _buildProductList(showOnlyCritical: false),
        ),
      ],
    ),
  ),
  Padding(
    padding: EdgeInsets.all(isMobile ? 12 : 20),
    child: Column(
      children: [
        _buildFilters(),
        const SizedBox(height: 12),
        Expanded(
          child: _buildProductList(showOnlyCritical: true),
        ),
      ],
    ),
  ),
],
),
);
}

```

```

Widget _buildQuickActions() {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Acciones rápidas',
            style: Theme.of(context).textTheme.titleMedium?.copyWith(
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 12),
          Wrap(
            spacing: 12,
            runSpacing: 12,
            children: [
              FilledButton.icon(
                onPressed: widget.onNavigateToSales,
                icon: const Icon(Icons.receipt_long),
                label: const Text('Nueva venta'),
              ),
              OutlinedButton.icon(
                onPressed: () => _openProductForm(),
                icon: const Icon(Icons.add),
                label: const Text('Registrar producto'),
              ),
            ],
          ),
        ],
      ),
    ),
  );
}

```

```

    ),
    ],
  ),
  ],
),
);
}

```

```

Widget _buildFilters() {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: LayoutBuilder(
        builder: (context, constraints) {
          final isMobile = constraints.maxWidth < 600;

          if (isMobile) {
            return _buildMobileFilters();
          }

          return Wrap(
            spacing: 12,
            runSpacing: 12,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              SizedBox(
                width: 280,
                child: _buildSearchField(),
              ),
              _dropdownContainer(
                label: 'Categoría',
                child: _buildCategoryDropdown(),
              ),
              _dropdownContainer(
                label: 'Estado',
                child: _buildStatusDropdown(),
              ),
              _dropdownContainer(
                label: 'Ordenar por',
                child: _buildSortDropdown(),
              ),
              IconButton.filledTonal(
                tooltip:
                  _ascending ? 'Orden ascendente' : 'Orden descendente',
                onPressed: () {
                  setState(() {
                    _ascending = !_ascending;
                  });
                  _reloadInventory();
                },
              ),
            ],
          );
        },
      ),
    ),
  );
}

```

```

    },
    icon: Icon(
      _ascending ? Icons.arrow_upward : Icons.arrow_downward,
    ),
  ),
  IconButton(
    tooltip: 'Actualizar',
    onPressed: _reloadInventory,
    icon: const Icon(Icons.refresh),
  ),
],
);
},
),
);
}

```

```

Widget _buildMobileFilters() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      _buildSearchField(),
      const SizedBox(height: 12),
      Row(
        children: [
          Expanded(
            child: _dropdownContainer(
              label: 'Categoría',
              child: _buildCategoryDropdown(),
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: _dropdownContainer(
              label: 'Estado',
              child: _buildStatusDropdown(),
            ),
          ),
        ],
      ),
      const SizedBox(height: 12),
      Row(
        children: [
          Expanded(
            child: _dropdownContainer(
              label: 'Ordenar por',
              child: _buildSortDropdown(),
            ),
          ),
        ],
      ),
    ],
  );
}

```

```

const SizedBox(width: 12),
IconButton.filledTonal(
  tooltip: _ascending ? 'Orden ascendente' : 'Orden descendente',
  onPressed: () {
    setState(() {
      _ascending = !_ascending;
    });
    _reloadInventory();
  },
  icon: Icon(
    _ascending ? Icons.arrow_upward : Icons.arrow_downward,
  ),
),
IconButton(
  tooltip: 'Actualizar',
  onPressed: _reloadInventory,
  icon: const Icon(Icons.refresh),
),
],
),
],
);
}

```

```

Widget _buildSearchField() {
  return TextField(
    controller: _searchController,
    onChanged: (_) {
      _reloadInventory();
    },
    decoration: InputDecoration(
      labelText: 'Buscar por nombre o SKU',
      prefixIcon: const Icon(Icons.search),
      border: const OutlineInputBorder(),
      suffixIcon: _searchController.text.isEmpty
        ? null
        : IconButton(
            onPressed: () {
              _searchController.clear();
              _reloadInventory();
            },
            icon: const Icon(Icons.clear),
          ),
    ),
  );
}

```

```

Widget _buildCategoryDropdown() {
  return DropdownButton<String>(
    value: _selectedCategory,

```

```

isExpanded: true,
underline: const SizedBox.shrink(),
items: [
  const DropdownMenuItem(
    value: "",
    child: Text('Todas'),
  ),
  ..._categories.map(
    (category) => DropdownMenuItem(
      value: category,
      child: Text(category),
    ),
  ),
],
onChanged: (value) {
  setState(() {
    _selectedCategory = value ?? "";
  });
  _reloadInventory();
},
);
}

```

```

Widget _buildStatusDropdown() {
  return DropdownButton<String>(
    value: _selectedStatus,
    isExpanded: true,
    underline: const SizedBox.shrink(),
    items: const [
      DropdownMenuItem(
        value: 'active',
        child: Text('Activos'),
      ),
      DropdownMenuItem(
        value: 'inactive',
        child: Text('Inactivos'),
      ),
      DropdownMenuItem(
        value: 'all',
        child: Text('Todos'),
      ),
    ],
    onChanged: (value) {
      setState(() {
        _selectedStatus = value ?? 'active';
      });
      _reloadInventory();
    },
  );
}

```

```

Widget _buildSortDropdown() {
  return DropdownButton<ProductSortOption>(
    value: _sortOption,
    isExpanded: true,
    underline: const SizedBox.shrink(),
    items: const [
      DropdownMenuItem(
        value: ProductSortOption.name,
        child: Text('Nombre'),
      ),
      DropdownMenuItem(
        value: ProductSortOption.price,
        child: Text('Precio'),
      ),
      DropdownMenuItem(
        value: ProductSortOption.stock,
        child: Text('Stock'),
      ),
    ],
    onChanged: (value) {
      if (value == null) return;
      setState(() {
        _sortOption = value;
      });
      _reloadInventory();
    },
  );
}

```

```

Widget _dropdownContainer({
  required String label,
  required Widget child,
}) {
  return InputDecorator(
    decoration: InputDecoration(
      labelText: label,
      border: const OutlineInputBorder(),
      contentPadding: const EdgeInsets.symmetric(
        horizontal: 12,
        vertical: 4,
      ),
    ),
    child: child,
  );
}

```

```

Widget _buildProductList({required bool showOnlyCritical}) {
  if (_isLoading) {
    return const Center(

```

```

        child: CircularProgressIndicator(),
      );
    }

    final displayProducts = showOnlyCritical
      ? _products.where((product) {
        final alert = _alerts[product.id];
        return product.isActive &&
          alert != null &&
          alert.isActive &&
          product.stock <= alert.minimumQuantity;
      }).toList()
      : _products;

    if (displayProducts.isEmpty) {
      return const Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(
              Icons.inventory_2_outlined,
              size: 72,
            ),
            SizedBox(height: 12),
            Text(
              'No se encontraron productos.',
              style: TextStyle(fontSize: 18),
            ),
          ],
        ),
      );
    }
  }

  return LayoutBuilder(
    builder: (context, constraints) {
      final columns = constraints.maxWidth >= 1100
        ? 3
        : constraints.maxWidth >= 700
          ? 2
          : 1;

      if (columns == 1) {
        return ListView.builder(
          padding: const EdgeInsets.only(bottom: 90),
          itemCount: displayProducts.length,
          itemBuilder: (context, index) {
            final product = displayProducts[index];

            return ProductCard(
              product: product,

```



```

        productId: 'PROD-001',
        minimumQuantity: 10,
    );

    final triggeredResult = await service.getTriggeredAlerts();

    expect(triggeredResult.isSuccess, true);
    expect(triggeredResult.data!.length, 1);
    expect(triggeredResult.data!.first.productId, 'PROD-001');

    // Creamos otro producto con stock 20 y alerta 5 (no debería
    dispararse)
    const p2 = Product(
      id: 'PROD-002',
      sku: 'SKU-002',
      name: 'Product 2',
      price: 50.0,
      stock: 20,
      category: 'Test',
      location: 'A2',
    );
    await productRepository.save(p2);
    await service.createAlert(
      productId: 'PROD-002',
      minimumQuantity: 5,
    );

    final triggeredResult2 = await service.getTriggeredAlerts();
    expect(triggeredResult2.data!.length, 1); // Sigue siendo sólo el
    primero
  });
}

```

Resultados Obtenidos

```

~/codigoSprints/paperboost ) flutter test test/logic/services/stock_alert_service_test.dart --plain-name "Obtener alertas disparadas (triggered)"
00:03 +1: All tests passed!

```

RQF-022: Notificar alertas de stock

Como se implementó

Archivo principal a testear: lib\logic\observers\stock_change_notifier.dart

```
import '../data/models/product.dart';
import 'stock_observer.dart';

/// Implementa el patrón Observer para notificaciones de cambios de stock.
///
/// Este notificador es un Singleton que gestiona observadores interesados
/// en cambios de inventario. Cuando el stock de un producto cambia,
/// notifica automáticamente a todos los observadores registrados.
class StockChangeNotifier {
  StockChangeNotifier._internal();

  static final StockChangeNotifier _instance =
    StockChangeNotifier._internal();

  static StockChangeNotifier get instance => _instance;

  final List<StockObserver> _observers = [];

  void attach(StockObserver observer) {
    if (!_observers.contains(observer)) {
      _observers.add(observer);
    }
  }

  void detach(StockObserver observer) {
    _observers.remove(observer);
  }

  void notifyStockChanged(
    Product product,
    int oldStock,
    int newStock,
  ) {
    for (final observer in _observers) {
      observer.onStockChanged(product, oldStock, newStock);
    }
  }

  void notifyStockUnavailable(Product product) {
    for (final observer in _observers) {
      observer.onStockUnavailable(product);
    }
  }
}
```

```
void clearObservers() {  
    _observers.clear();  
}  
  
int get observerCount => _observers.length;  
}
```

Pruebas Realizadas

```
group('StockChangeNotifier', () {  
    late StockChangeNotifier notifier;  
  
    setUp(() {  
        notifier = StockChangeNotifier.instance;  
        notifier.clearObservers();  
    });  
  
    tearDown(() {  
        notifier.clearObservers();  
    });  
  
    const product = Product(  
        id: 'PROD-001',  
        sku: 'SKU-001',  
        name: 'Test',  
        price: 100.0,  
        stock: 50,  
        category: 'Test',  
        location: 'A1',  
    );  
  
    test('observerCount es 0 al inicio', () {  
        expect(notifier.observerCount, 0);  
    });  
  
    test('attach agrega observador', () {  
        final observer = TestStockObserver();
```

```
    notifier.attach(observer);
    expect(notifier.observerCount, 1);
  });

test('attach no duplica observadores', () {
  final observer = TestStockObserver();
  notifier.attach(observer);
  notifier.attach(observer);
  expect(notifier.observerCount, 1);
});

test('detach remove observador', () {
  final observer = TestStockObserver();
  notifier.attach(observer);
  expect(notifier.observerCount, 1);
  notifier.detach(observer);
  expect(notifier.observerCount, 0);
});

test('notifyStockChanged notifica a observadores', () {
  final observer = TestStockObserver();
  notifier.attach(observer);

  notifier.notifyStockChanged(product, 50, 30);

  expect(observer.stockChangedCalls, hasLength(1));
  expect(observer.lastChangedProduct!.id, 'PROD-001');
  expect(observer.lastOldStock, 50);
  expect(observer.lastNewStock, 30);
});

test('notifyStockUnavailable notifica a observadores', () {
  final observer = TestStockObserver();
  notifier.attach(observer);

  notifier.notifyStockUnavailable(product);

  expect(observer.stockUnavailableCalls, hasLength(1));
  expect(observer.lastUnavailableProduct!.id, 'PROD-001');
```

```

});

test('notifica a múltiples observadores', () {
    final observer1 = TestStockObserver();
    final observer2 = TestStockObserver();
    notifier.attach(observer1);
    notifier.attach(observer2);

    notifier.notifyStockChanged(product, 10, 5);

    expect(observer1.stockChangedCalls, hasLength(1));
    expect(observer2.stockChangedCalls, hasLength(1));
});

test('clearObservers remueve todos los observadores', () {
    notifier.attach(TestStockObserver());
    notifier.attach(TestStockObserver());
    expect(notifier.observerCount, 2);

    notifier.clearObservers();
    expect(notifier.observerCount, 0);
});

test('no notifica a observadores removidos', () {
    final observer = TestStockObserver();
    notifier.attach(observer);
    notifier.detach(observer);

    notifier.notifyStockChanged(product, 10, 5);

    expect(observer.stockChangedCalls, isEmpty);
});

group('StockChangeNotifier - Singleton', () {
    test('instance es singleton', () {
        final instance1 = StockChangeNotifier.instance;
        final instance2 = StockChangeNotifier.instance;
        expect(identical(instance1, instance2), true);
    });
});

```

```
    });  
  });  
}
```

Resultados Obtenidos

```
.../codigoSprints/paperboost ) flutter test test/logic/observers/stock_change_notifier_test.dart  
00:03 +10: All tests passed!
```

RQF-023: Dar de baja un producto

Como se implementó

Archivo principal a testear: lib\logic\services\product_service.dart

```
import '../data/models/product.dart';  
import '../data/repositories/product_repository.dart';  
import '../results/operation_result.dart';  
import '../validators/product_validator.dart';  
  
enum ProductSortOption {  
  name,  
  price,  
  stock,  
}  
  
class ProductService {  
  ProductService({  
    required ProductRepository productRepository,  
  }) : _productRepository = productRepository;  
  
  final ProductRepository _productRepository;  
  
  Future<OperationResult<Product>> registerProduct({  
    required String sku,  
    required String name,  
    required double price,  
    required int stock,  
    required String category,  
    required String location,  
  }) async {  
    final product = Product(  

```

```

        id: _generateProductId(),
        sku: sku.trim().toUpperCase(),
        name: name.trim(),
        price: price,
        stock: stock,
        category: category.trim(),
        location: location.trim(),
    );

    final validationErrors =
        ProductValidator.validateProduct(product);

    if (validationErrors.isNotEmpty) {
        return OperationResult<Product>.failure(
            message: validationErrors.join("\n"),
        );
    }

    final duplicatedSku =
        await _productRepository.existsBySku(product.sku);

    if (duplicatedSku) {
        return OperationResult<Product>.failure(
            message:
                'Ya existe un producto registrado con el SKU ${product.sku}.',
        );
    }

    try {
        await _productRepository.save(product);

        return OperationResult<Product>.success(
            message: 'Producto registrado correctamente.',
            data: product,
        );
    } catch (error) {
        return OperationResult<Product>.failure(
            message: 'No se pudo registrar el producto: $error',
        );
    }
}

Future<OperationResult<List<Product>>> getProducts({
    bool includeInactive = false,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: includeInactive,
        );
    }
}

```

```

        return OperationResult<List<Product>>.success(
            message: products.isEmpty
                ? 'No existen productos registrados.'
                : 'Productos consultados correctamente.',
            data: products,
        );
    } catch (error) {
        return OperationResult<List<Product>>.failure(
            message: 'No se pudo consultar el inventario: $error',
        );
    }
}

Future<OperationResult<Product>> getProductById(
    String id,
) async {
    if (id.trim().isEmpty) {
        return OperationResult<Product>.failure(
            message: 'El identificador del producto es obligatorio.',
        );
    }

    final product = await _productRepository.findById(
        id.trim(),
    );

    if (product == null) {
        return OperationResult<Product>.failure(
            message: 'Producto no encontrado.',
        );
    }

    return OperationResult<Product>.success(
        message: 'Producto encontrado.',
        data: product,
    );
}

Future<OperationResult<List<Product>>> searchProducts({
    String query = "",
    String? category,
    ProductStatus? status,
    ProductSortOption sortOption =
        ProductSortOption.name,
    bool ascending = true,
}) async {
    try {
        final products = await _productRepository.findAll(
            includeInactive: true,
        );
    }
}

```

```

final normalizedQuery =
    query.trim().toLowerCase();

final filteredProducts = products.where((product) {
    final matchesQuery =
        normalizedQuery.isEmpty ||
        product.name
            .toLowerCase()
            .contains(normalizedQuery) ||
        product.sku
            .toLowerCase()
            .contains(normalizedQuery);

    final matchesCategory =
        category == null ||
        category.trim().isEmpty ||
        product.category.toLowerCase() ==
            category.trim().toLowerCase();

    final matchesStatus =
        status == null || product.status == status;

    return matchesQuery &&
        matchesCategory &&
        matchesStatus;
}).toList();

_sortProducts(
    filteredProducts,
    sortOption: sortOption,
    ascending: ascending,
);

return OperationResult<List<Product>>.success(
    message: filteredProducts.isEmpty
        ? 'No se encontraron productos.'
        : 'Búsqueda realizada correctamente.',
    data: filteredProducts,
);
} catch (error) {
    return OperationResult<List<Product>>.failure(
        message: 'No se pudo realizar la búsqueda: $error',
    );
}
}

Future<OperationResult<Product>> updateProduct({
    required Product product,
}) async {

```

```
final currentProduct =
    await _productRepository.findById(product.id);

if (currentProduct == null) {
    return OperationResult<Product>.failure(
        message: 'El producto que desea editar no existe.',
    );
}

final normalizedProduct = product.copyWith(
    sku: product.sku.trim().toUpperCase(),
    name: product.name.trim(),
    category: product.category.trim(),
    location: product.location.trim(),
);

final validationErrors =
    ProductValidator.validateProduct(
        normalizedProduct,
    );

if (validationErrors.isNotEmpty) {
    return OperationResult<Product>.failure(
        message: validationErrors.join('\n'),
    );
}

final duplicatedSku =
    await _productRepository.existsBySku(
        normalizedProduct.sku,
        excludingProductId: normalizedProduct.id,
    );

if (duplicatedSku) {
    return OperationResult<Product>.failure(
        message:
            'Ya existe otro producto con el SKU ${normalizedProduct.sku}.',
    );
}

try {
    await _productRepository.update(
        normalizedProduct,
    );

    return OperationResult<Product>.success(
        message: 'Producto actualizado correctamente.',
        data: normalizedProduct,
    );
} catch (error) {
```

```

        return OperationResult<Product>.failure(
            message: 'No se pudo actualizar el producto: $error',
        );
    }
}

Future<OperationResult<Product>> deactivateProduct(
    String id,
) async {
    final product = await _productRepository.findById(id);

    if (product == null) {
        return OperationResult<Product>.failure(
            message:
                'El producto que desea dar de baja no existe.',
        );
    }

    if (!product.isActive) {
        return OperationResult<Product>.failure(
            message: 'El producto ya se encuentra dado de baja.',
        );
    }

    try {
        await _productRepository.deactivate(id);

        final inactiveProduct = product.copyWith(
            status: ProductStatus.inactive,
        );

        return OperationResult<Product>.success(
            message: 'Producto dado de baja exitosamente.',
            data: inactiveProduct,
        );
    } catch (error) {
        return OperationResult<Product>.failure(
            message: 'No se pudo dar de baja el producto: $error',
        );
    }
}

void _sortProducts(
    List<Product> products, {
    required ProductSortOption sortOption,
    required bool ascending,
}) {
    products.sort((first, second) {
        int comparison;

```

```

switch (sortOption) {
    case ProductSortOption.name:
        comparison = first.name
            .toLowerCase()
            .compareTo(second.name.toLowerCase());
        break;

    case ProductSortOption.price:
        comparison =
            first.price.compareTo(second.price);
        break;

    case ProductSortOption.stock:
        comparison =
            first.stock.compareTo(second.stock);
        break;
}

return ascending ? comparison : -comparison;
});
}

String _generateProductId() {
    final timestamp =
        DateTime.now().microsecondsSinceEpoch;

    return 'PRODUCT-$timestamp';
}
}

```

Pruebas Realizadas

```

group('ProductService - RQF-023: Dar de Baja Producto', () {
    late ProductService productService;

    setUp(() async {
        final repo = InMemoryProductRepository();
        productService = ProductService(productRepository: repo);
    });

    test('dar de baja producto activo', () async {
        final saved = await productService.registerProduct(

```

```
        sku: 'SKU-001',
        name: 'Producto',
        price: 10.0,
        stock: 10,
        category: 'Cat',
        location: 'A1',
    );

    final result = await productService.deactivateProduct(
        saved.data!.id,
    );
    expect(result.isSuccess, true);
    expect(result.data!.status, ProductStatus.inactive);
    expect(result.data!.isActive, false);
});

test('rechazar baja de producto inexistente', () async {
    final result = await productService.deactivateProduct('NO-EXISTE');
    expect(result.isSuccess, false);
    expect(result.message, contains('no existe'));
});

test('rechazar baja de producto ya inactivo', () async {
    final saved = await productService.registerProduct(
        sku: 'SKU-001',
        name: 'Producto',
        price: 10.0,
        stock: 10,
        category: 'Cat',
        location: 'A1',
    );

    await productService.deactivateProduct(saved.data!.id);
    final result = await productService.deactivateProduct(
        saved.data!.id,
    );
    expect(result.isSuccess, false);
    expect(result.message, contains('ya se encuentra dado de baja'));
});
```

```
});
```

Resultados Obtenidos

```
~/codigoSprints/paperboost ) flutter test test/logic/services/product_service_test.dart --plain-name "ProductService - RQF-023: Dar de Baja Producto"
00:02 +3: All tests passed!
```

RQF-024: Confirmar baja del producto

Como se implementó

Archivo principal a testear: lib\presentation\pages\inventory_page.dart

```
import 'package:flutter/material.dart';

import '../data/models/product.dart';
import '../data/models/stock_alert.dart';
import '../logic/controllers/auth_controller.dart';
import '../logic/controllers/product_controller.dart';
import '../logic/controllers/stock_alert_controller.dart';
import '../logic/observers/stock_observer.dart';
import '../logic/observers/stock_change_notifier.dart';
import '../logic/results/operation_result.dart';
import '../logic/services/product_service.dart';
import '../widgets/product_card.dart';
import 'product_form_page.dart';

class InventoryPage extends StatefulWidget {
  const InventoryPage({
    required this.authController,
    required this.productController,
    required this.stockAlertController,
    this.onLogout,
    this.onNavigateToSales,
    super.key,
  });

  final AuthController authController;
  final ProductController productController;
  final StockAlertController stockAlertController;
  final VoidCallback? onLogout;
  final VoidCallback? onNavigateToSales;
```

```

@override
State<InventoryPage> createState() => _InventoryPageState();
}

class _InventoryPageState extends State<InventoryPage>
    with SingleTickerProviderStateMixin
    implements StockObserver {
    final _searchController = TextEditingController();
    late TabController _tabController;

    List<Product> _products = [];
    List<String> _categories = [];
    Map<String, StockAlert> _alerts = {};

    String _selectedCategory = "";
    String _selectedStatus = 'active';

    ProductSortOption _sortOption = ProductSortOption.name;

    bool _ascending = true;
    bool _isLoading = true;

    @override
    void initState() {
        super.initState();
        _tabController = TabController(length: 2, vsync: this);
        StockChangeNotifier.instance.attach(this);
        _reloadInventory();
    }

    @override
    void dispose() {
        _tabController.dispose();
        StockChangeNotifier.instance.detach(this);
        _searchController.dispose();
        super.dispose();
    }

    @override
    void onStockChanged(Product product, int oldStock, int newStock) {
        if (mounted) {
            _reloadInventory();
        }
    }

    @override
    void onStockUnavailable(Product product) {
        if (mounted) {
            _reloadInventory();
        }
    }
}

```

```
}
```

```
ProductStatus? get _statusFilter {  
  switch (_selectedStatus) {  
    case 'active':  
      return ProductStatus.active;  
    case 'inactive':  
      return ProductStatus.inactive;  
    default:  
      return null;  
  }  
}
```

```
Future<void> _reloadInventory() async {  
  setState(() {  
    _isLoading = true;  
  });  
  
  final allResult = await widget.productController.getProducts(  
    includeInactive: true,  
  );  
  
  final categories = <String>{};  
  
  for (final product in allResult.data ?? <Product>[]) {  
    categories.add(product.category);  
  }  
  
  final searchResult = await widget.productController.searchProducts(  
    query: _searchController.text,  
    category: _selectedCategory.isEmpty ? null : _selectedCategory,  
    status: _statusFilter,  
    sortOption: _sortOption,  
    ascending: _ascending,  
  );  
  
  final alertsResult = await widget.stockAlertController.getAlerts();  
  final Map<String, StockAlert> loadedAlerts = {};  
  for (final alert in alertsResult.data ?? <StockAlert>[]) {  
    loadedAlerts[alert.productId] = alert;  
  }  
  
  if (!mounted) {  
    return;  
  }  
  
  setState(() {  
    _categories = categories.toList()..sort();  
    _products = searchResult.data ?? <Product>[];  
    _alerts = loadedAlerts;  
  });  
}
```

```

    _isLoading = false;
  });
}

Future<void> _openProductForm({
  Product? product,
}) async {
  final changed = await Navigator.push<bool>(
    context,
    MaterialPageRoute(
      builder: (_) => ProductFormPage(
        productController: widget.productController,
        product: product,
      ),
    ),
  );

  if (changed == true) {
    await _reloadInventory();
  }
}

Future<void> _confirmDeactivate(
  Product product,
) async {
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (dialogContext) {
      return AlertDialog(
        title: const Text('Confirmar baja'),
        content: Text(
          '¿Desea dar de baja el producto '
          '"${product.name}"?',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(
                dialogContext,
                false,
              );
            },
            child: const Text('Cancelar'),
          ),
          FilledButton(
            onPressed: () {
              Navigator.pop(
                dialogContext,
                true,
              );
            },

```

```

        },
        child: const Text('Confirmar'),
      ),
    ],
  );
},
);

if (confirmed != true) {
  return;
}

final result = await widget.productController.deactivateProduct(product.id);

if (!mounted) {
  return;
}

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text(result.message),
    backgroundColor:
      result.isSuccess ? null : Theme.of(context).colorScheme.error,
  ),
);

if (result.isSuccess) {
  await _reloadInventory();
}
}

void _openConfigureAlert(Product product) async {
  final alert = _alerts[product.id];
  final isNew = alert == null;

  final quantityController = TextEditingController(
    text: alert?.minimumQuantity.toString() ?? '5',
  );
  bool isActive = alert?.isActive ?? true;

  final result = await showDialog<bool>(
    context: context,
    builder: (dialogContext) {
      return StatefulBuilder(
        builder: (context, setStateDialog) {
          return AlertDialog(
            title: Text('Configurar Alerta - ${product.name}'),
            content: SingleChildScrollView(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.min,

```

```

        children: [
          const Text(
            'Define la cantidad mínima para notificar cuando las existencias alcancen o
bajen de este límite.',
            style: TextStyle(fontSize: 13),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: quantityController,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
              labelText: 'Cantidad Mínima',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          SwitchListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('Alerta Activa'),
            subtitle:
              const Text('Habilita/deshabilita notificaciones'),
            value: isActive,
            onChanged: (val) {
              setStateDialog(() {
                isActive = val;
              });
            },
          ),
        ],
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(dialogContext, false),
        child: const Text('Cancelar'),
      ),
      FilledButton(
        onPressed: () => Navigator.pop(dialogContext, true),
        child: const Text('Guardar'),
      ),
    ],
  );
},
);
},
);

if (result != true) return;

final minQty = int.tryParse(quantityController.text) ?? -1;

```

```

if (minQty < 0) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content:
          Text('Cantidad mínima inválida. Debe ser mayor o igual a 0.'),
        backgroundColor: Colors.red,
      ),
    );
  }
  return;
}

OperationResult<StockAlert> opResult;
if (isNew) {
  opResult = await widget.stockAlertController.createAlert(
    productId: product.id,
    minimumQuantity: minQty,
  );
} else {
  opResult = await widget.stockAlertController.updateAlert(
    id: alert.id,
    minimumQuantity: minQty,
    isActive: isActive,
  );
}

if (mounted) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(opResult.message),
      backgroundColor: opResult.isSuccess ? Colors.green : Colors.red,
    ),
  );
  if (opResult.isSuccess) {
    _reloadInventory();
  }
}

void _showTriggeredAlertsDialog() {
  final criticalProducts = _products.where((product) {
    final alert = _alerts[product.id];
    return product.isActive &&
      alert != null &&
      alert.isActive &&
      product.stock <= alert.minimumQuantity;
  }).toList();

  showDialog(

```

```

context: context,
builder: (context) {
  return AlertDialog(
    title: Row(
      children: const [
        Icon(Icons.warning, color: Colors.amber),
        SizedBox(width: 8),
        Expanded(
          child: Text(
            'Alertas de Stock Crítico',
            overflow: TextOverflow.ellipsis,
          ),
        ),
      ],
    ),
    content: criticalProducts.isEmpty
      ? const Text(
          'No hay productos con existencias críticas actualmente.')
      : SizedBox(
          width: double.maxFinite,
          child: ListView.builder(
            shrinkWrap: true,
            itemCount: criticalProducts.length,
            itemBuilder: (context, index) {
              final product = criticalProducts[index];
              final alert = _alerts[product.id]!;
              return Padding(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Text(
                  '⚠️ ${product.name} - SKU: ${product.sku} (Stock: ${product.stock}, Mín:
                    ${alert.minimumQuantity})',
                  style: const TextStyle(fontSize: 12),
                  overflow: TextOverflow.ellipsis,
                ),
              );
            },
          ),
        ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cerrar'),
      ),
    ],
  );
}
);
}

```

```
Widget _buildNotificationBell() {
```

```

final criticalCount = _products.where((product) {
  final alert = _alerts[product.id];
  return product.isActive &&
    alert != null &&
    alert.isActive &&
    product.stock <= alert.minimumQuantity;
}).length;

if (criticalCount == 0) {
  return const IconButton(
    icon: Icon(Icons.notifications_none),
    onPressed: null,
  );
}

return Badge(
  label: Text(criticalCount.toString()),
  backgroundColor: Colors.red,
  child: IconButton(
    icon: const Icon(Icons.notifications_active, color: Colors.amber),
    onPressed: _showTriggeredAlertsDialog,
  ),
);
}

void _logout() {
  widget.onLogout?.call();
}

@override
Widget build(BuildContext context) {
  final isMobile = MediaQuery.of(context).size.width < 600;

  return Scaffold(
    appBar: AppBar(
      title: const Text('Inventario PaperBoost'),
      bottom: TabBar(
        controller: _tabController,
        tabs: const [
          Tab(icon: Icon(Icons.inventory), text: 'Todos'),
          Tab(icon: Icon(Icons.warning_amber), text: 'Crítico'),
        ],
      ),
    actions: [
      _buildNotificationBell(),
      if (isMobile)
        PopupMenuButton<String>(
          icon: const Icon(Icons.more_vert),
          onPressed: (value) {
            if (value == 'logout') _logout();
          },
        ),
    ],
  );
}

```

```

    },
    itemBuilder: (_) => [
      PopupMenuItem(
        value: 'logout',
        child: Row(
          children: const [
            Icon(Icons.logout, size: 20),
            SizedBox(width: 8),
            Expanded(child: Text('Cerrar sesión')),
          ],
        ),
      ),
    ],
  ),
)
else ...[
  IconButton(
    tooltip: 'Nueva venta',
    onPressed: widget.onNavigateToSales,
    icon: const Icon(Icons.receipt_long),
  ),
  IconButton(
    tooltip: 'Cerrar sesión',
    onPressed: _logout,
    icon: const Icon(Icons.logout),
  ),
],
),
floatingActionButton: isMobile
? FloatingActionButton(
  onPressed: () => _openProductForm(),
  child: const Icon(Icons.add),
)
: FloatingActionButton.extended(
  onPressed: () => _openProductForm(),
  icon: const Icon(Icons.add),
  label: const Text('Registrar producto'),
),
body: TabBarView(
  controller: _tabController,
  children: [
    Padding(
      padding: EdgeInsets.all(isMobile ? 12 : 20),
      child: Column(
        children: [
          if (!isMobile) ...[
            _buildQuickActions(),
            const SizedBox(height: 16),
          ],
          _buildFilters(),
        ],
      ),
    ),
  ],
),

```

```

        const SizedBox(height: 12),
        Expanded(
          child: _buildProductList(showOnlyCritical: false),
        ),
      ],
    ),
  ),
  Padding(
    padding: EdgeInsets.all(isMobile ? 12 : 20),
    child: Column(
      children: [
        _buildFilters(),
        const SizedBox(height: 12),
        Expanded(
          child: _buildProductList(showOnlyCritical: true),
        ),
      ],
    ),
  ),
],
),
);
}

```

```

Widget _buildQuickActions() {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Acciones rápidas',
            style: Theme.of(context).textTheme.titleMedium?.copyWith(
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 12),
          Wrap(
            spacing: 12,
            runSpacing: 12,
            children: [
              FilledButton.icon(
                onPressed: widget.onNavigateToSales,
                icon: const Icon(Icons.receipt_long),
                label: const Text('Nueva venta'),
              ),
              OutlinedButton.icon(
                onPressed: () => _openProductForm(),
                icon: const Icon(Icons.add),
              ),
            ],
          ),
        ],
      ),
    ),
  );
}

```

```

        label: const Text('Registrar producto'),
      ),
    ],
  ),
],
),
);
}

```

```

Widget _buildFilters() {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: LayoutBuilder(
        builder: (context, constraints) {
          final isMobile = constraints.maxWidth < 600;

          if (isMobile) {
            return _buildMobileFilters();
          }

          return Wrap(
            spacing: 12,
            runSpacing: 12,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              SizedBox(
                width: 280,
                child: _buildSearchField(),
              ),
              _dropdownContainer(
                label: 'Categoría',
                child: _buildCategoryDropdown(),
              ),
              _dropdownContainer(
                label: 'Estado',
                child: _buildStatusDropdown(),
              ),
              _dropdownContainer(
                label: 'Ordenar por',
                child: _buildSortDropdown(),
              ),
              IconButton.filledTonal(
                tooltip:
                  _ascending ? 'Orden ascendente' : 'Orden descendente',
                onPressed: () {
                  setState(() {
                    _ascending = !_ascending;
                  });
                },
              ),
            ],
          );
        },
      ),
    ),
  );
}

```

```

        _reloadInventory();
    },
    icon: Icon(
      _ascending ? Icons.arrow_upward : Icons.arrow_downward,
    ),
  ),
  IconButton(
    tooltip: 'Actualizar',
    onPressed: _reloadInventory,
    icon: const Icon(Icons.refresh),
  ),
],
);
},
),
);
}

```

```

Widget _buildMobileFilters() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      _buildSearchField(),
      const SizedBox(height: 12),
      Row(
        children: [
          Expanded(
            child: _dropdownContainer(
              label: 'Categoría',
              child: _buildCategoryDropdown(),
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: _dropdownContainer(
              label: 'Estado',
              child: _buildStatusDropdown(),
            ),
          ),
        ],
      ),
      const SizedBox(height: 12),
      Row(
        children: [
          Expanded(
            child: _dropdownContainer(
              label: 'Ordenar por',
              child: _buildSortDropdown(),
            ),
          ),
        ],
      ),
    ],
  );
}

```

```

    ),
    const SizedBox(width: 12),
    IconButton.filledTonal(
      tooltip: _ascending ? 'Orden ascendente' : 'Orden descendente',
      onPressed: () {
        setState(() {
          _ascending = !_ascending;
        });
        _reloadInventory();
      },
      icon: Icon(
        _ascending ? Icons.arrow_upward : Icons.arrow_downward,
      ),
    ),
    IconButton(
      tooltip: 'Actualizar',
      onPressed: _reloadInventory,
      icon: const Icon(Icons.refresh),
    ),
  ],
),
],
);
}

```

```

Widget _buildSearchField() {
  return TextField(
    controller: _searchController,
    onChanged: (_) {
      _reloadInventory();
    },
    decoration: InputDecoration(
      labelText: 'Buscar por nombre o SKU',
      prefixIcon: const Icon(Icons.search),
      border: const OutlineInputBorder(),
      suffixIcon: _searchController.text.isEmpty
        ? null
        : IconButton(
            onPressed: () {
              _searchController.clear();
              _reloadInventory();
            },
            icon: const Icon(Icons.clear),
          ),
    ),
  );
}

```

```

Widget _buildCategoryDropdown() {
  return DropdownButton<String>(

```

```

value: _selectedCategory,
isExpanded: true,
underline: const SizedBox.shrink(),
items: [
  const DropdownMenuItem(
    value: "",
    child: Text('Todas'),
  ),
  ..._categories.map(
    (category) => DropdownMenuItem(
      value: category,
      child: Text(category),
    ),
  ),
],
onChanged: (value) {
  setState(() {
    _selectedCategory = value ?? "";
  });
  _reloadInventory();
},
);
}

```

```

Widget _buildStatusDropdown() {
  return DropdownButton<String>(
    value: _selectedStatus,
    isExpanded: true,
    underline: const SizedBox.shrink(),
    items: const [
      DropdownMenuItem(
        value: 'active',
        child: Text('Activos'),
      ),
      DropdownMenuItem(
        value: 'inactive',
        child: Text('Inactivos'),
      ),
      DropdownMenuItem(
        value: 'all',
        child: Text('Todos'),
      ),
    ],
    onChanged: (value) {
      setState(() {
        _selectedStatus = value ?? 'active';
      });
      _reloadInventory();
    },
  );
}

```

```
}
```

```
Widget _buildSortDropdown() {  
  return DropdownButton<ProductSortOption>(  
    value: _sortOption,  
    isExpanded: true,  
    underline: const SizedBox.shrink(),  
    items: const [  
      DropdownMenuItem(  
        value: ProductSortOption.name,  
        child: Text('Nombre'),  
      ),  
      DropdownMenuItem(  
        value: ProductSortOption.price,  
        child: Text('Precio'),  
      ),  
      DropdownMenuItem(  
        value: ProductSortOption.stock,  
        child: Text('Stock'),  
      ),  
    ],  
    onChanged: (value) {  
      if (value == null) return;  
      setState(() {  
        _sortOption = value;  
      });  
      _reloadInventory();  
    },  
  );  
}
```

```
Widget _dropdownContainer({  
  required String label,  
  required Widget child,  
}) {  
  return InputDecorator(  
    decoration: InputDecoration(  
      labelText: label,  
      border: const OutlineInputBorder(),  
      contentPadding: const EdgeInsets.symmetric(  
        horizontal: 12,  
        vertical: 4,  
      ),  
    ),  
    child: child,  
  );  
}
```

```
Widget _buildProductList({required bool showOnlyCritical}) {  
  if (!_isLoading) {
```

```

return const Center(
  child: CircularProgressIndicator(),
);
}

final displayProducts = showOnlyCritical
? _products.where((product) {
  final alert = _alerts[product.id];
  return product.isActive &&
    alert != null &&
    alert.isActive &&
    product.stock <= alert.minimumQuantity;
}).toList()
: _products;

if (displayProducts.isEmpty) {
return const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Icon(
        Icons.inventory_2_outlined,
        size: 72,
      ),
      SizedBox(height: 12),
      Text(
        'No se encontraron productos.',
        style: TextStyle(fontSize: 18),
      ),
    ],
  ),
);
}

return LayoutBuilder(
  builder: (context, constraints) {
    final columns = constraints.maxWidth >= 1100
      ? 3
      : constraints.maxWidth >= 700
        ? 2
        : 1;

    if (columns == 1) {
      return ListView.builder(
        padding: const EdgeInsets.only(bottom: 90),
        itemCount: displayProducts.length,
        itemBuilder: (context, index) {
          final product = displayProducts[index];

          return ProductCard(

```

```

        product: product,
        stockAlert: _alerts[product.id],
        onConfigureAlert: () => _openConfigureAlert(product),
        onEdit: () => _openProductForm(
            product: product,
        ),
        onDeactivate: () => _confirmDeactivate(product),
    );
  },
);
}

```

```

return GridView.builder(
  padding: const EdgeInsets.only(bottom: 90),
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: columns,
    crossAxisSpacing: 12,
    mainAxisSpacing: 12,
    childAspectRatio: 1.55,
  ),
  itemCount: displayProducts.length,
  itemBuilder: (context, index) {
    final product = displayProducts[index];

    return ProductCard(
      product: product,
      stockAlert: _alerts[product.id],
      onConfigureAlert: () => _openConfigureAlert(product),
      onEdit: () => _openProductForm(
        product: product,
      ),
      onDeactivate: () => _confirmDeactivate(product),
    );
  },
);
},
);
},
);
}
}
}

```

Pruebas Realizadas

```

test('dar de baja producto activo', () async {
  final saved = await productService.registerProduct(
    sku: 'SKU-001',
  );
});

```

```
        name: 'Producto',  
        price: 10.0,  
        stock: 10,  
        category: 'Cat',  
        location: 'A1',  
    );  
  
    final result = await productService.deactivateProduct(  
        saved.data!.id,  
    );  
    expect(result.isSuccess, true);  
    expect(result.data!.status, ProductStatus.inactive);  
    expect(result.data!.isActive, false);  
  });
```

Resultados Obtenidos

```
.../codigoSprints/paperboost > flutter test test/logic/services/product_service_test.dart --plain-name "dar de baja producto activo"  
00:02 +1: All tests passed!
```

5. PROBLEMAS Y SOLUCIONES

5.1 Problema identificado

Durante la ejecución de las pruebas se identificaron los siguientes problemas:

Problema

Warnings de hit test en pruebas UI

Descripción

Varios tests en sales_page_test.dart mostraban advertencias del tipo "A call to tap() with finder [...] derived an Offset that would not hit test on the specified widget". Esto ocurría

al intentar hacer tap sobre widgets como "Selecciona un producto" que estaban fuera de la zona de interacción visible (offscreen u obstruidos por otros widgets).

Solución

Ajustar los finders para que apunten a widgets realmente visibles y accesibles, o agregar "warnIfMissed: false" explícitamente en los tap() cuando el comportamiento esperado sea intencional. También se puede aumentar el tamaño de la ventana de prueba o hacer scroll antes de interactuar.

6. CONCLUSIONES

6.1 Logros Alcanzados

- Se implementaron 428 pruebas automatizadas distribuidas en 35 archivos, cubriendo los 18 requisitos funcionales del sistema.
- El 100% de las pruebas pasaron exitosamente, demostrando que la lógica de negocio, validaciones, controladores, repositorios y la interfaz de usuario funcionan conforme a lo esperado.
- Se logró una cobertura por capas completa: modelos, repositorios en memoria, servicios, controladores, validadores, páginas y widgets.
- Se estableció una suite de pruebas ejecutable en 45 segundos, permitiendo retroalimentación rápida durante el desarrollo.
- Se implementaron pruebas para funcionalidades críticas como autenticación, registro y consulta de productos, generación de notas de venta con validación de stock, y administración de alertas de stock mínimo.

6.2 Calidad del Software

La suite de pruebas arroja los siguientes indicadores de calidad:

- Total de pruebas: 428 tests distribuidos en 35 archivos y 74 suites (grupos)
- Tasa de éxito: 100% — todos los tests pasaron en una ejecución de 45 segundos
- Cobertura por capas: Las pruebas abarcan todas las capas de la arquitectura:
- Modelos (data/models): validación de estructura de datos (Product, Sale, StockAlert, etc.)
- Repositorios (data/repositories): operaciones CRUD en memoria
- Servicios (logic/services): lógica de negocio principal con 6 archivos de servicio probados
- Controladores (logic/controllers): orquestación y delegación
- Validadores (logic/validators): reglas de validación de entrada
- Presentación (presentation/pages/, presentation/widgets/): integración de UI con 7 archivos de prueba
- Observadores/Sesión/Seguridad: infraestructura transversal
- Cobertura funcional: Los requisitos funcionales (RF) cuentan con pruebas asociadas, todos con tasa de éxito del 100%
- Framework: flutter_test con el ecosistema estándar de Flutter/Dart, lo que garantiza compatibilidad y mantenibilidad

6.3 Lecciones Aprendidas

- **Pruebas por capas vs. pruebas integradas:** el enfoque actual prueba cada capa de forma aislada (servicios con repositorios mock, controladores con servicios mock, UI con controladores mock). Si bien esto es correcto para tests unitarios, flujos completos como "registrar producto → consultar inventario → editar → dar de baja" no se prueban de extremo a extremo.
- **La cobertura cuantitativa no garantiza cobertura cualitativa:** algunos RQFs tienen muchos tests, mientras que otros tienen pocos, pero ambos pasan al 100%. La cantidad no siempre refleja la robustez de las pruebas.

6.4 Recomendaciones Futuras

- **Corregir warnings de hit test:** Revisar y corregir los 11 warnings detectados en `sales_page_test.dart` para mantener una suite de pruebas libre de advertencias.
- **Agregar pruebas de integración:** Considerar incorporar `integration_test` para probar flujos completos en un entorno más realista (navegación entre pantallas, persistencia real).
- **Automatizar en CI/CD:** Implementar un pipeline de GitHub Actions u otro sistema que ejecute `flutter test --coverage` en cada push y genere reportes automáticos. Porar pruebas de regresión automatizadas en un pipeline de integración continua.